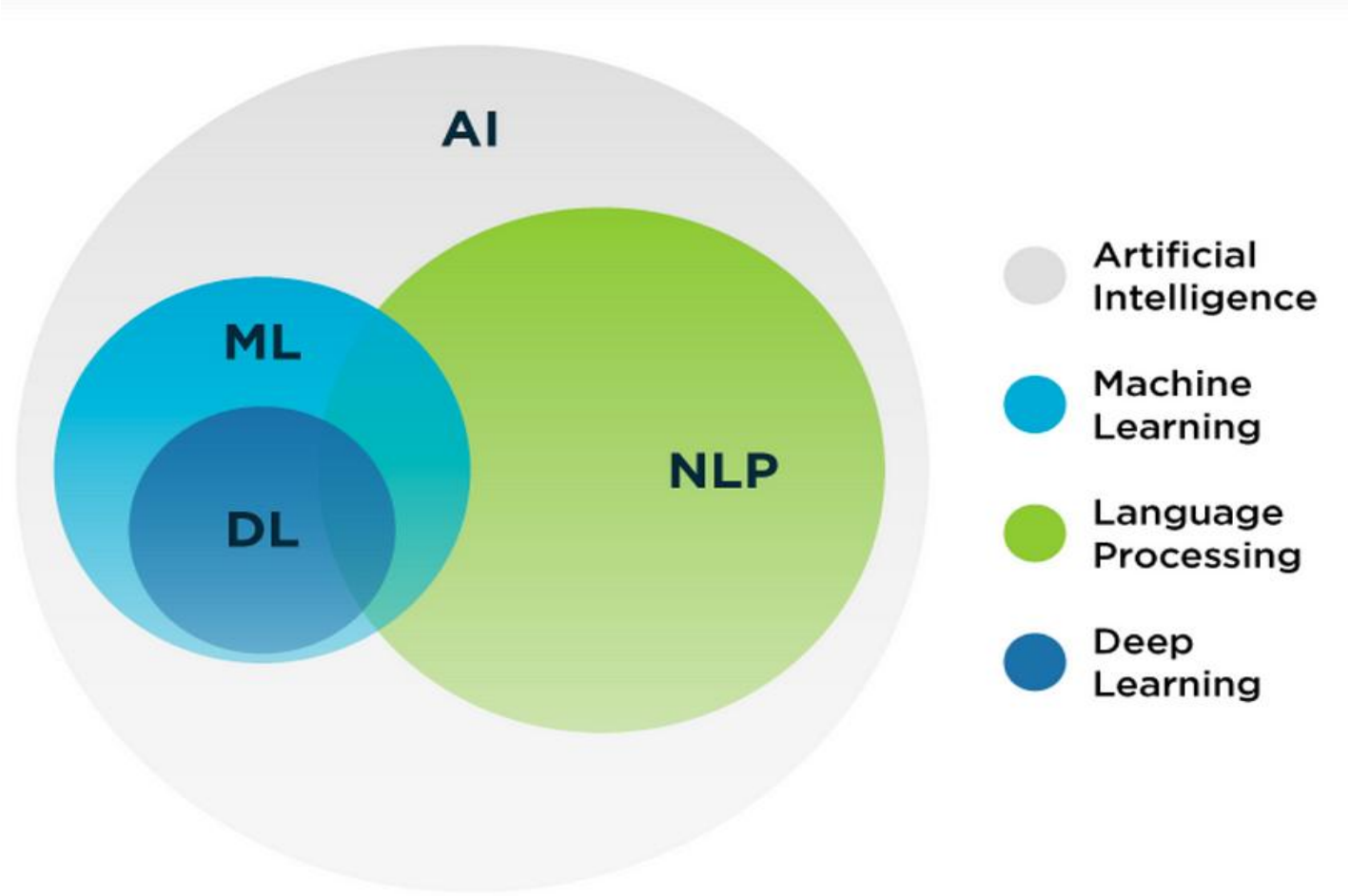
Introduction To NLP

Natural Language Processing (NLP) is a subfield of artificial intelligence (AI) That helps computers understand, interpret and manipulate human language.



Contents:

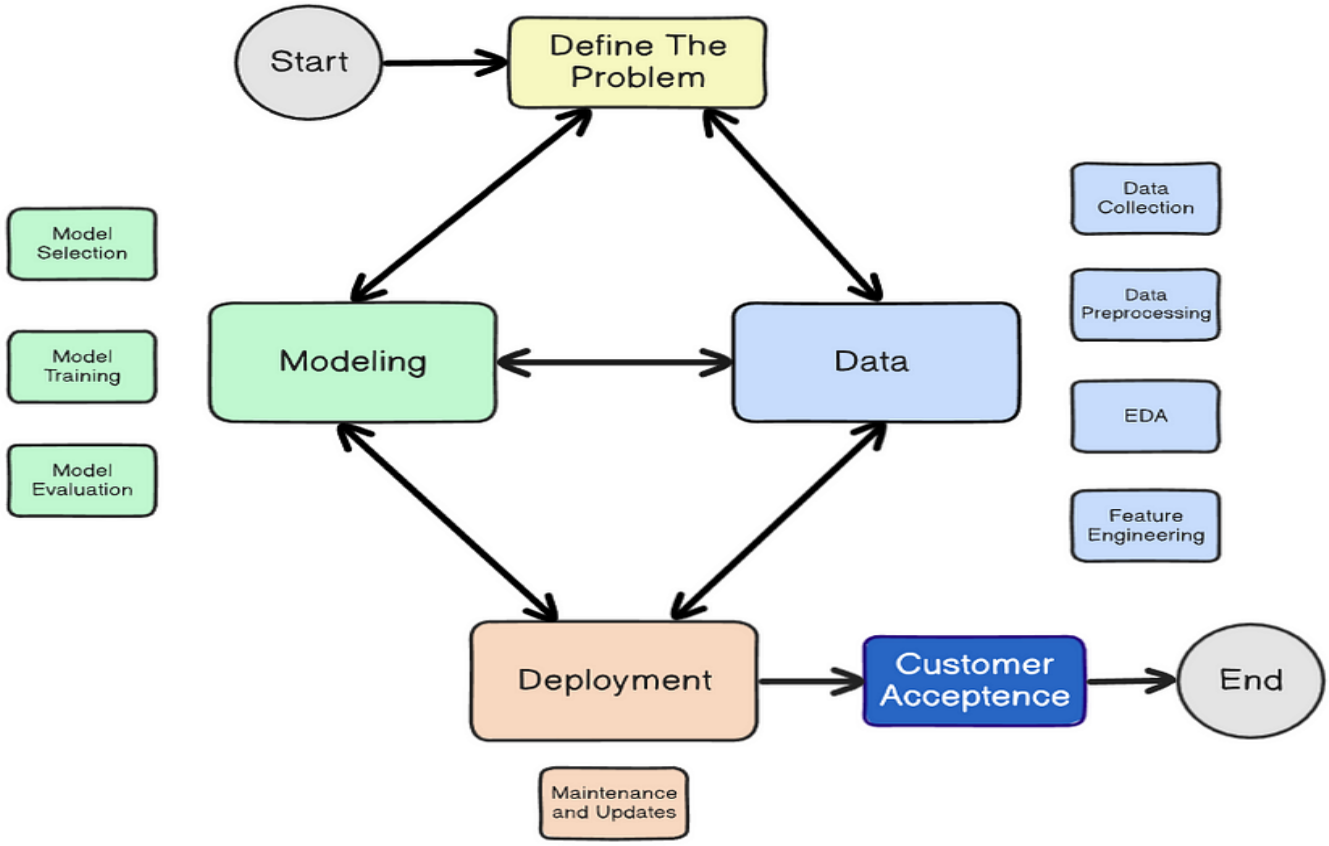
- Application Of NLP
- Steps Involved In NLP Projects
- Steps in Text Preprocessing
- Most Common Libraries For NLP

Applications Of NLP

1. **Named Entity Recognition (NER)**: Identifying and classifying named entities (people, organizations, locations, etc.) within the text.
2. **Part-of-Speech (POS) Tagging**: Identifying the grammatical parts of speech (nouns, verbs, adjectives, etc.) in a sentence.
3. **Search Engines**: Improving the relevance and accuracy of search results.

4. **Chatbots and Virtual Assistants**: Enabling automated, human-like interactions.
5. **Speech Recognition**: Converting spoken language into text.
6. **Sentiment Analysis**: Gauging public sentiment from social media, reviews, and feedback.
7. **Text Classification**: Categorizing text into predefined categories (e.g., spam detection).
8. **Machine Translation**: Translating text between languages.
9. **Information Extraction**: Pulling structured information from unstructured text.
10. **Text Summarization**: Creating summaries of documents or articles.
11. **Content Generation**: Automatically generating content (e.g., news articles, creative writing).

Steps Involved In NLP Projects



1. Define the Problem

- **Objective**: Clearly define the problem you are trying to solve or the goal you want to achieve with NLP
- **Examples**: Sentiment analysis, machine translation, chatbots, text classification

2. Data Collection

- **Objective**: Gather the text data required for the project
- **Sources**: Web scraping, APIs, public datasets, internal databases

3. Data Preprocessing

- **Objective**: Clean and prepare the text data for analysis.
- **Steps**:

Text Cleaning*: Remove or correct noisy data (e.g., HTML tags, special characters)*

Tokenization*: Split text into words or sentences.*

Normalization*: Convert text to a standard format (e.g., lowercase, removing punctuation).*

Stop Words Removal*: Remove common words that do not carry significant meaning (e.g., "the", "and").*

Stemming and Lemmatization*: Reduce words to their root forms.*

Handling Missing Data*: Deal with missing or incomplete data points.*

4. Exploratory Data Analysis (EDA)

- **Objective**: Understand the structure and characteristics of the data
- **Steps**:

Descriptive Statistics*: Analyze the distribution of words, sentence lengths, etc*

Visualization*: Use word clouds, bar charts, and other visual tools to gain insights*

Correlation Analysis*: Identify relationships between different features in the data*

5. Feature Engineering

- **Objective**: Create features that machine learning models will use
- **Techniques**:

***Bag of Words (BoW)**: Represent text as a collection of word counts*

***Term Frequency-Inverse Document Frequency (TF-IDF)**: Adjust word counts based on their importance*

***Word Embeddings**: Represent words as vectors in a continuous vector space (e.g., Word2Vec, GloVe)*

***Sentence Embeddings**: Represent sentences as vectors (e.g., BERT, Sentence-BERT)*

6. Model Selection

- **Objective**: Choose the appropriate model(s) for the task
- **Options**:

***Rule-Based Models**: Use predefined rules for specific tasks*

***Statistical Models**: Apply statistical methods to analyze text*

***Machine Learning Models**: Use algorithms like Naive Bayes, SVM, or logistic regression*

***Deep Learning Models**: Utilize neural networks such as RNNs, LSTMs, GRUs, Bidirectoinal RNN, Sequence-to-Sequence(Encoder/decoder), Transformers (Attention mechanism), Encoder-only Transformers (BERT), Decoder-only Transformers (GPT) or Transfer Learning*

7. Model Training

- **Objective**: Train the selected model(s) on the prepared data
- **Steps**:

***Split Data**: Divide data into training, validation, and test sets*

***Training**: Fit the model to the training data*

***Hyperparameter Tuning**: Adjust model parameters to optimize performance*

***Validation**: Evaluate the model on the validation set and adjust as needed*

8. Model Evaluation

- **Objective**: Assess the performance of the trained model.
- Metrics:

***Precision, Recall, F1-Score**: Metrics for evaluating classification models*

***BLEU Score**: Evaluating the quality of machine-translated text*

***ROUGE Score**: Evaluating the quality of text summarization*

9. Model Deployment

- **Objective**: Deploy the model to a production environment
- **Steps**:

***Export Model**: Save the trained model in a suitable format (e.g., pickle)*

***Integration**: Integrate the model into the application or system*

***Monitoring**: Continuously monitor the model's performance and update as needed*

10. Maintenance and Updates

- **Objective**: Ensure the model remains effective over time
- **Steps**:

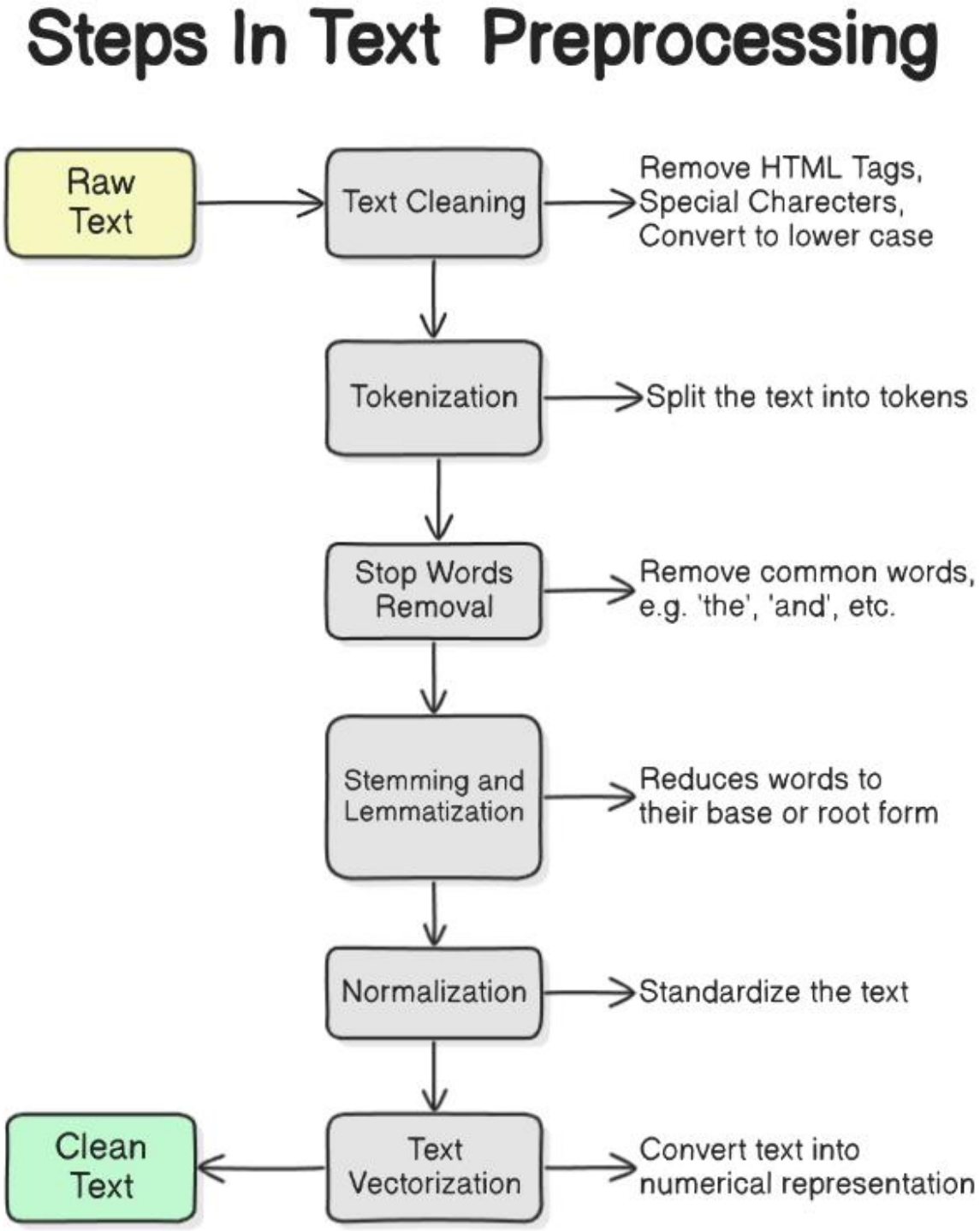
***Regular Updates**: Retrain the model with new data to maintain accuracy*

***Monitoring**: Track model performance and user feedback*

***Scalability**: Ensure the model can handle increased load and new data*

Steps In Text Preprocessing

Text preprocessing is a crucial step in Natural Language Processing (NLP) projects. It involves transforming raw text into a format that is suitable for analysis and modeling.



1. Text Cleaning

- **Objective**: Remove unwanted characters and clean the text to make it suitable for analysis
- **Steps**:

Remove HTML Tags*: Eliminate HTML tags if the text is scraped from the web*

Remove Special Characters*: Remove punctuation, special characters, and digits if they are not needed*

Convert to Lowercase*: Convert all text to lowercase to maintain consistency*

2. Tokenization

- **Objective**: Split the text into individual tokens (words, phrases, or sentences)
- **Types**:

Word Tokenization*: Split text into words*

Sentence Tokenization*: Split text into sentences*

- **Tools**: NLTK, spaCy

3. Stop Words Removal

- **Objective**: Remove common words that do not carry significant meaning and are often considered noise
- **Examples**: “the”, “and”, “is”, “in”, etc
- **Tools**: NLTK, spaCy

4. Stemming and Lemmatization

- **Objective**: Reduce words to their root forms to ensure consistency in text analysis
- **Stemming**: Reduces words to their base or root form, often resulting in non-dictionary words (removing suffixes from a word)
- **Example**: “running” -> “run”
- **Lemmatization**: Reduces words to their base or root form using a dictionary, resulting in dictionary words (consider the context and convert the word to it is meaningful base form)
- **Example**: “better” -> “good”
- **Tools**: NLTK, spaCy

5. Normalization

- **Objective**: Standardize text for uniformity and consistency
- **Steps**:

Lowercasing*: Convert all characters to lowercase*

Removing Accents*: Remove diacritical marks (e.g., “café” to “cafe”)*

***Expanding Contractions**: Convert contractions to their full forms (e.g., "don't" to "do not")*

***Replacing Numbers**: Replace numerical values with a specific token (e.g., "num")*

Map the words to fixed language word (e.g. "b4" to before and "ttyl" to talk to you later)

6. Text Augmentation (Optional)

- **Objective**: Create additional training data by slightly modifying existing text
- **Techniques**:

***Synonym Replacement**: Replace words with their synonyms*

***Random Insertion**: Insert random words at random positions*

***Random Swap**: Swap the position of two words in a sentence*

***Random Deletion**: Randomly delete words from the sentence*

7. Removing Rare Words and Frequent Words (Optional)

- **Objective**: Remove words that appear too infrequently or too frequently to help improve model performance
- **Steps**:

***Rare Words**: Remove words that occur below a certain frequency threshold*

***Frequent Words**: Remove words that occur above a certain frequency threshold*

8. Text Vectorization

- **Objective**: Convert text into numerical representation for machine learning models
- **Techniques**:

***Bag of Words (BoW)**: Represent text as a collection of word counts*

***Term Frequency-Inverse Document Frequency (TF-IDF)**: Adjust word counts based on their importance in the corpus*

***Word Embeddings**: Represent words as dense vectors in a continuous vector space (e.g., Word2Vec, GloVe)*

***Sentence Embeddings**: Represent sentences as dense vectors (e.g., BERT embeddings)*

9. Handling Imbalanced Data (Optional)

- **Objective**: Address class imbalance in the dataset.
- **Techniques**:

Undersampling: *Reduce the number of instances in the majority class*

Oversampling: *Increase the number of instances in the minority class*

Synthetic Data Generation: *Use techniques like SMOTE to create synthetic samples*

Most Common Python Libraries for NLP

1. NLTK (Natural Language Toolkit)

- **Description**: A comprehensive library for text processing and analysis
- **Features**:

Tokenization

POS tagging

Stemming and lemmatization

Parsing

Named Entity Recognition (NER)

Sentiment analysis

Text classification

Extensive corpus support

- **Usage**: Great for educational purposes and prototyping

2. spaCy

- **Description**: An industrial-strength NLP library designed for performance and ease of use
- **Features**:

Tokenization

POS tagging

Named Entity Recognition (NER)

Dependency parsing

Text classification

Word vectors and embeddings

Support for multiple languages

- **Usage**: Suitable for production environments

3. Gensim

- **Description**: A library for topic modeling and document similarity
- **Features**:

Word2Vec

FastText

Doc2Vec

Latent Dirichlet Allocation (LDA)

TF-IDF

- **Usage**: Excellent for working with large text corpora and unsupervised learning tasks.

4. Transformers (Hugging Face)

- **Description**: A library that provides state-of-the-art transformer models
- **Features**:

Pre-trained models for text classification, NER, translation, summarization, and question answering

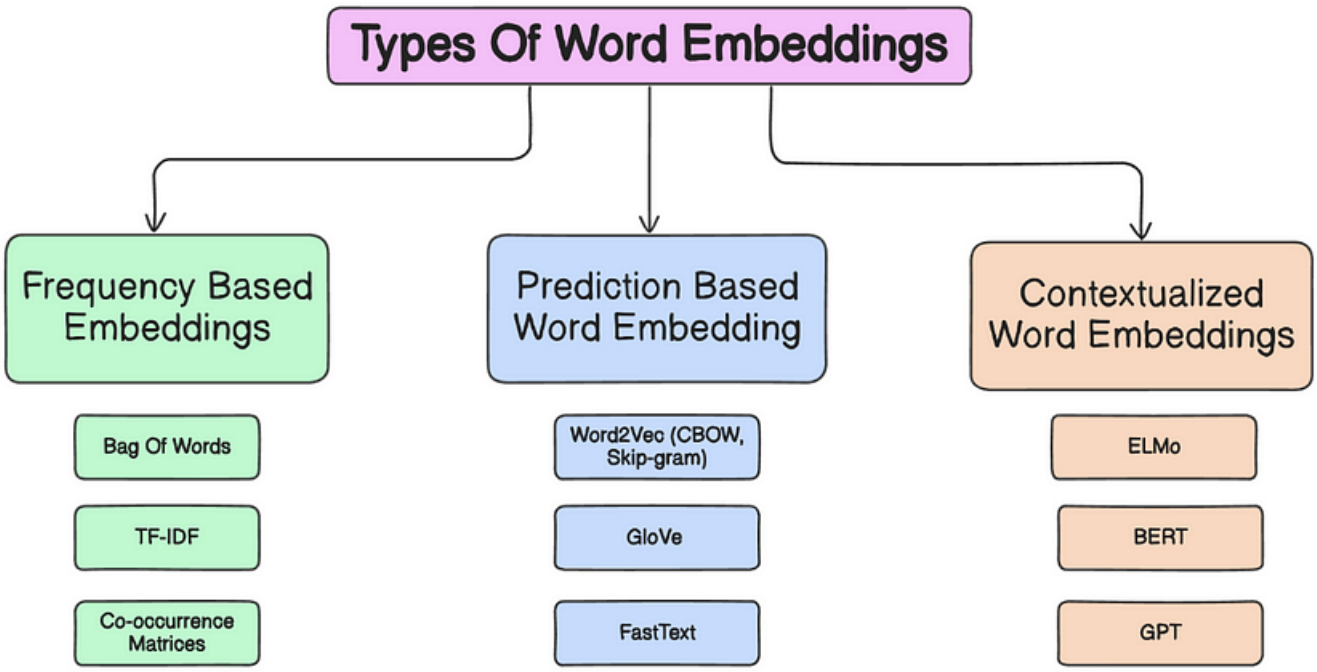
BERT, GPT, T5, and other transformer architectures

Fine-tuning and inference capabilities

- **Usage**: Widely used for leveraging cutting-edge transformer-based models

Word Embedding Techniques

- It is a technique which converts words into vectors
- Word embeddings are the richer representation of the relationship between tokens
- Word embeddings transform the high-dimensional and sparse representations of words (like one-hot encodings) into dense, lower-dimensional vectors that encode meaningful linguistic information



Contents:

- **1. Frequency-based Embedding:** BOW, TF-IDF, Co-occurrence Matrics
- **2. Prediction Based Word Embedding:** Word2Vec, Glove, FastText
- **3. Contextualized Word Embedding:** ELMO, BERT, GPT, T5

1. Frequency-based Embeddings

- Frequency-based word embeddings are a category of techniques that represent words based on their frequency of occurrence and co-occurrence in a given corpus
- Unlike prediction-based methods that learn embeddings through prediction tasks, frequency-based methods derive embeddings from the statistical properties of the text

1-Count Vectorization (Bag of Words)

- **Description**: This method creates a vector for each document by counting the number of times each word appears

How Bag of Words Works

1. **Tokenization**: Split the text into individual words or tokens.
2. **Vocabulary Creation**: Create a list of all unique words (vocabulary) from the text corpus.
3. **Vectorization**: Convert each document into a vector of numbers where each number represents the count (or frequency) of a word in the document.

Practical Example:

- Below is our text corpus:

No.	Sentences
1	"Welcome to UK, UK is a good place."
2	"In the UK, Oxford Uni is the best."

Note: First of all, we must perform text preprocessing, such as converting to lowercase, removing stop words and etc.

- Below is our text corpus after preprocessing:

No.	Sentences
1	"welcome uk uk good place"
2	"uk oxford uni best"

Step1: Tokenization

Split each sentence into words:

- Sentence 1: [“welcome”, “uk”, “uk”, “good”, “place”]
- Sentence 2: [“uk”, “oxford”, “uni”, “best”]

Step2: Vocabulary Creation

Create a list of unique words from the entire corpus (ignoring case and punctuation):

- Vocabulary: [“welcome”, “uk”, “good”, “place”, “oxford”, “uni”, “best”]

Step3: Vectorization

Convert each sentence into a vector based on the word count:

- sentence 1: [1, 2, 1, 1, 0, 0, 0]
- sentence 2: [0, 1, 0, 0, 1, 1, 1]

sentences	welcome	uk	good	place	oxford	uni	best
sentence 1	1	2	1	1	0	0	0
sentence 2	0	1	0	0	1	1	1

```
# python implementatoin
from sklearn.feature_extraction.text import CountVectorizer
corpus = [
    "Welcome to UK, UK is a good place.",
    "In the UK, Oxford Uni is the best."
]
# Create an instance of CountVectorizer
vectorizer = CountVectorizer()
```

```
# Fit the model and transform the corpus into BoW vectors
X = vectorizer.fit_transform(corpus)
# Display the vocabulary and vectors
print("Vocabulary:", vectorizer.get_feature_names_out())
print("Vectors:\n", X.toarray())
```

- **Advantages**: Simple and easy to understand
- **Disadvantages**: Results in high-dimensional, sparse vectors that do not capture semantic relationships between words

2-TF-IDF (Term Frequency-Inverse Document Frequency)

- **Description**: TF-IDF adjusts the raw word count by considering the importance of words in the context of the entire corpus
- **Components**:
- **Term Frequency (TF)**: Measures how frequently a word appears in a document
- **Inverse Document Frequency (IDF)**: Measures how important a word is by considering how often it appears across all documents
- **Formula**:

$$TF - IDF = TF \times IDF$$

$$TF = \frac{\text{No of repetation of words in sentences}}{\text{No of words in sentence}}$$

$$IDF = log(\frac{\text{No of sentences}}{\text{No of sentences containing the word}})$$

Practical Example:

- Below is our text corpus after preprocessing:

No.	Sentences
1	"welcome uk uk good place"
2	"uk oxford uni best"

Step1: Compute TF

- Vocabulary: [“welcome”, “uk”, “good”, “place”, “oxford”, “uni”, “best”]

Voc	sentence1	sentence2
welcome	1/5	0
uk	2/5	1/4
good	1/5	0
place	1/5	0
oxford	0	1/4
uni	0	1/4
best	0	1/4

Step2: Compute IDF

Vocabulary	IDF
welcome	log(2/1)
uk	log(2/3)
good	log(2/1)
place	log(2/1)
oxford	log(2/1)
uni	log(2/1)
best	log(2/1)

Step3: Compute TF x IDF

sentences	welcome	uk	good	place	oxford	uni	best
sentence 1	1/5 x log(2/1)	2/5 x log(2/3)	1/5 x log(2/1)	1/5 x log(2/1)	0	0	0
sentence 2	0	1/4 x log(2/3)	0	0	1/4 x log(2/1)	1/4 x log(2/1)	1/4 x log(2/1)

```
# python implementation
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "Welcome to UK, UK is a good place.",
    "In the UK, Oxford Uni is the best."
]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)
# Get feature names (vocabulary)
feature_names = vectorizer.get_feature_names_out()
# Convert the TF-IDF matrix to a dense array
tfidf_matrix = X.toarray()
```

```
# Display the results
print("Vocabulary:", feature_names)
print("TF-IDF Vectors:\n", tfidf_matrix)
```

- **Advantages**: Reduces the impact of commonly occurring words that are less informative
- **Disadvantages**: It still results in sparse vectors and does not capture word semantics

3-Co-occurrence Matrices

- **Description**: Represent words by their co-occurrence with other words within a specific context window

How Co-occurrence Matrices Work

1. **Define Context Window**: Determine the window size around each word to consider its neighboring words.
2. **Count Co-occurrences**: For each word pair within the context window, count how often they appear together.

Practical Example:

- Below is our text corpus after preprocessing:

No.	Sentences
1	"welcome uk uk good place"
2	"uk oxford uni best"

- Vocabulary: [“welcome”, “uk”, “good”, “place”, “oxford”, “uni”, “best”]

Step 1: Define Context Window Size

- A context window size of 1 (considering one word to the left and one word to the right).

Step 2: Count Co-occurrences

1st Iteration:

- Focus word: welcome
- Window Length = 1

- Context words: uk → after ‘welcome’ and there is no word before

2nd Iteration:

- Focus word: uk
- Window Length = 1
- Context words: uk → after ‘uk’ and welcome word before ‘uk’

And it continues …

_	welcome	uk	good	place	oxford	uni	best
welcome	0	1	0	0	0	0	0
uk	1	2	1	0	1	0	0
good	0	1	0	1	0	0	0
place	0	0	1	0	0	0	0
oxford	0	1	0	0	0	1	0
uni	0	0	0	0	1	0	1
best	0	0	0	0	0	1	0

- **Advantages**: Captures some semantic relationships by considering word co-occurrences
- **Disadvantages**: High-dimensional and sparse; can be computationally expensive to compute for large corpora

2. Prediction-based Embeddings

- Prediction-based word embeddings are techniques that create word representations by predicting the context in which words appear
- These methods use neural network models to learn word vectors that capture semantic relationships between words based on their usage patterns in large text corpora

1-Word2Vec

Word2Vec is one of the most well-known prediction-based embedding techniques, developed at Google.

Models:

- **Continuous Bag of Words (CBOW)**: Predicts the target word based on its surrounding context words. For example, given the context words “the quick brown”, it predicts the target word “fox”
- **Skip-gram**: Predicts the surrounding context words given the target word. For example, given the target word “fox”, it predicts context words like “the”, “quick”, “brown”

Training: Both models use a neural network with a single hidden layer to learn embeddings. The hidden layer weights become the word vectors.

Advantages: Efficient to train on large corpora, captures semantic relationships well.

2-GloVe (Global Vectors for Word Representation)

GloVe, developed by researchers at Stanford, combines the benefits of both global matrix factorization (like LSA) and local context window methods (like Word2Vec).

- **Method**: Constructs a word co-occurrence matrix from the corpus and then uses matrix factorization to learn word vectors. It captures the global statistical information of the corpus.

Training: Minimizes a weighted least squares objective that models the logarithm of the probability of word co-occurrences.

Advantages: Combines the strengths of word co-occurrence statistics and prediction-based learning.

3-FastText

Developed by Facebook’s AI Research (FAIR) lab, FastText extends Word2Vec by taking sub word information into account.

- **Method**: Represents words as bags of character n-grams. For example, the word “where” could be represented by character trigrams like “whe”, “her”, “ere”.

Training: Similar to Word2Vec, but the model predicts the target word using the sum of its subword n-gram vectors.

Advantages: Handles out-of-vocabulary words better, and captures morphological information.

3. Contextualized word embeddings

- Contextualized word embeddings represent a significant advancement in NLP, enabling models to understand and generate language with greater nuance and accuracy
- By taking context into account, these embeddings provide more meaningful word representations, leading to superior performance across a wide range of NLP tasks

1- ELMo (Embeddings from Language Models)

Developed by: Allen Institute for AI.

Architecture: Deep bidirectional LSTM network.

Training Objective: Language modeling, predicting the next word in a sequence.

Pros:

- **Context-Sensitive**: Generates different embeddings for the same word based on its context.
- **Pre-trained on Large Corpus**: Can be fine-tuned for various NLP tasks, improving performance.

Cons:

- **Computationally Intensive**: Requires significant computational resources for training and inference.
- **Complexity**: More complex to implement and fine-tune compared to static embeddings.

2-BERT (Bidirectional Encoder Representations from Transformers)

Developed by: Google AI.

Architecture: Transformer-based model.

Training Objective: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

Pros:

- **Bidirectional Context**: Considers both left and right context, providing rich embeddings.
- **State-of-the-Art Performance**: Excels in various NLP benchmarks and tasks.

Cons:

- **Resource-Intensive**: Requires substantial memory and processing power.
- **Training Time**: Long training times due to the complexity of the model and the amount of data required.

3-GPT (Generative Pretrained Transformer)

Developed by: OpenAI.

Architecture: Transformer decoder.

Training Objective: Language modeling, predicting the next word in a sequence.

Pros:

- **Strong Performance in Text Generation**: Excels at generating coherent and contextually appropriate text.
- **Unidirectional Context**: Effective for tasks where left-to-right context is crucial.

Cons:

- **Unidirectional Limitation**: This does not consider right-to-left context, which can be a limitation for some tasks.
- **High Computational Demand**: Similar to BERT, it requires significant computational resources.

4-T5 (Text-To-Text Transfer Transformer)

Developed by: Google Research.

Architecture: Transformer architecture.

Training Objective: Converts all NLP tasks into a text-to-text format.

Pros:

- **Versatility**: Can handle a wide range of NLP tasks with a unified approach.
- **Strong Performance**: Achieves excellent results across multiple benchmarks.

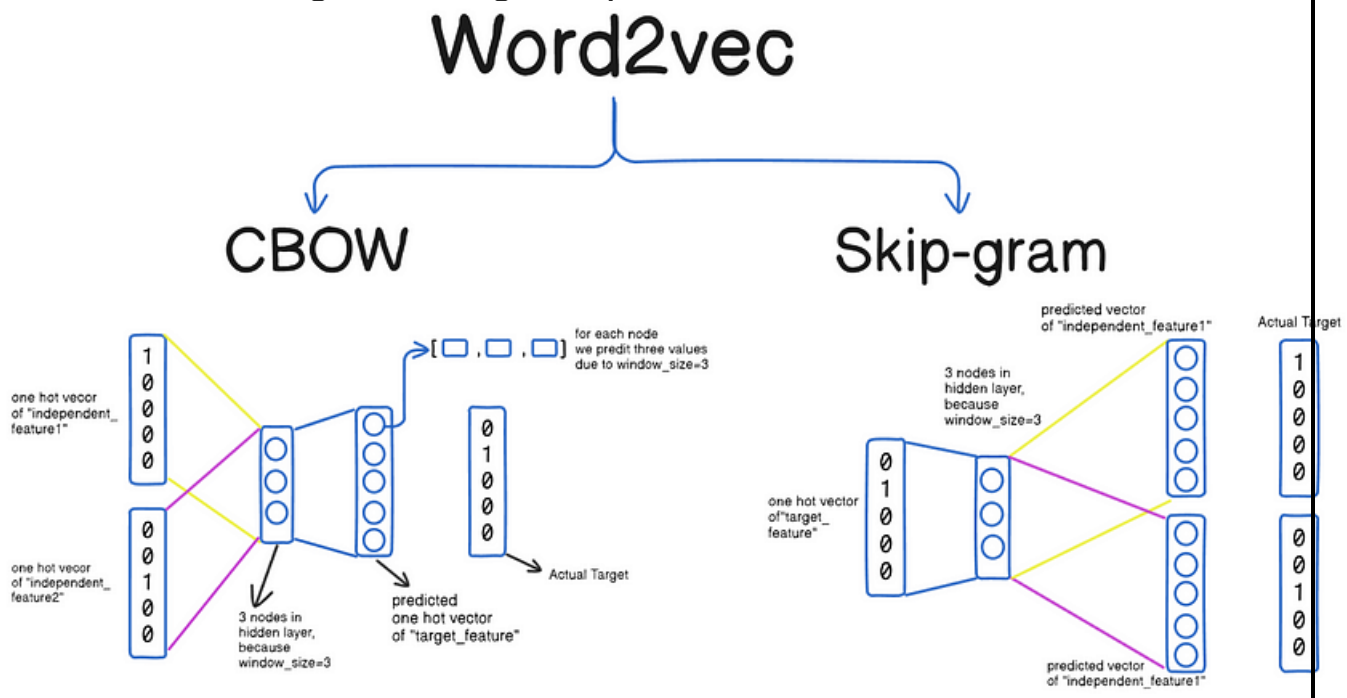
Cons:

- **Resource-Intensive**: High computational requirements for training and inference.

- **Complexity**: Implementing and fine-tuning can be challenging due to the model's flexibility and size.

Word2Vec (CBOW, Skip-gram)

- Word2Vec is an important model for natural language processing (NLP) developed by researchers at Google.
- Word2Vec is a group of related models used to produce word embeddings, which are dense vector representations of words in a continuous vector space.
- A two-layer network to generate word embedding given a text corpus.
- These embeddings capture semantic relationships between words based on their usage in a large corpus of text.



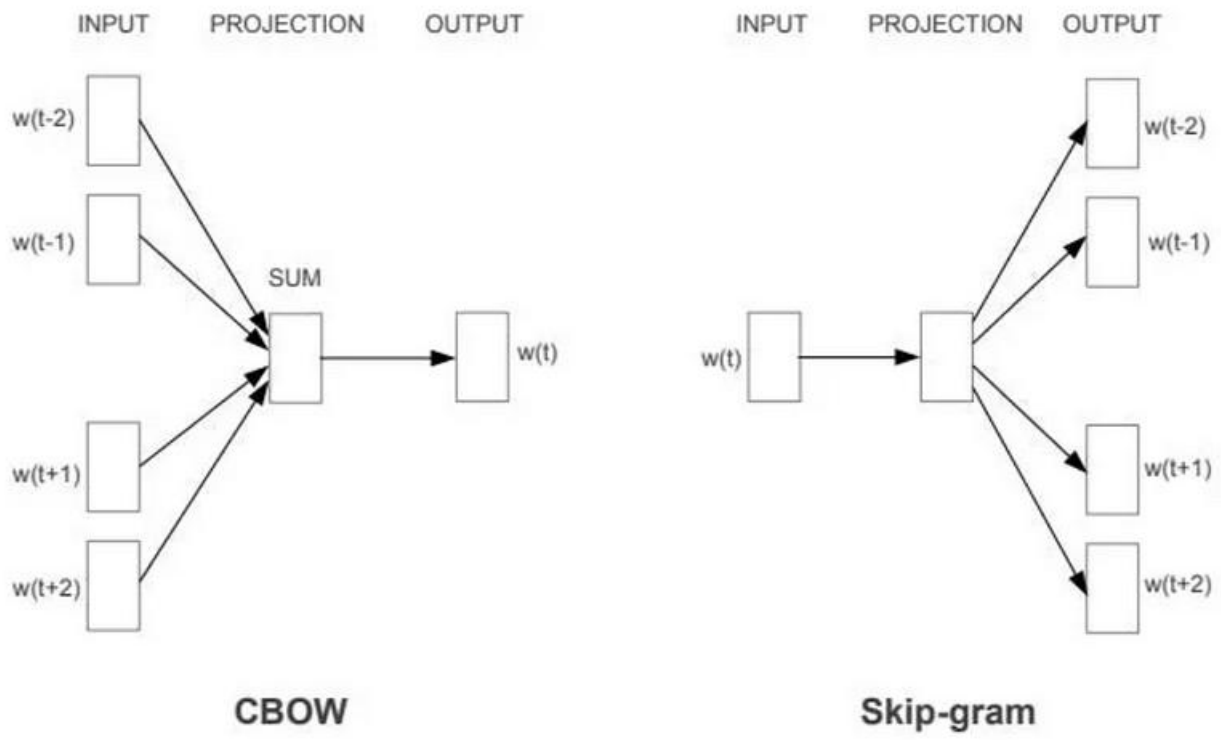
Word Embeddings: Word embeddings are fixed-size, dense vectors representing words. They capture semantic meaning in such a way that

similar words have similar vectors. For example, “king” and “queen” might have vectors that are close to each other.

Architecture

Word2Vec has two main architectures for generating word embeddings:

1. **Continuous Bag-of-Words (CBOW)**
2. **Skip-Gram**



1. Continuous Bag-of-Words (CBOW)

- Predict the target word (center word) from the surrounding context words.
- The model averages the vectors of context words and uses this average to predict the target word.

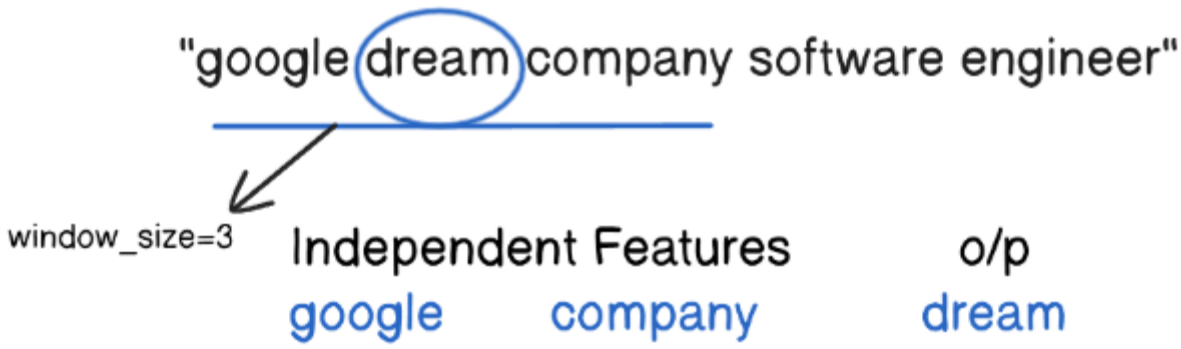
- Faster to train since it predicts only one word from multiple context words.

Practical Example:

- For simplicity, imagine we got these five words: “google dream company software engineer”

1st Iteration:

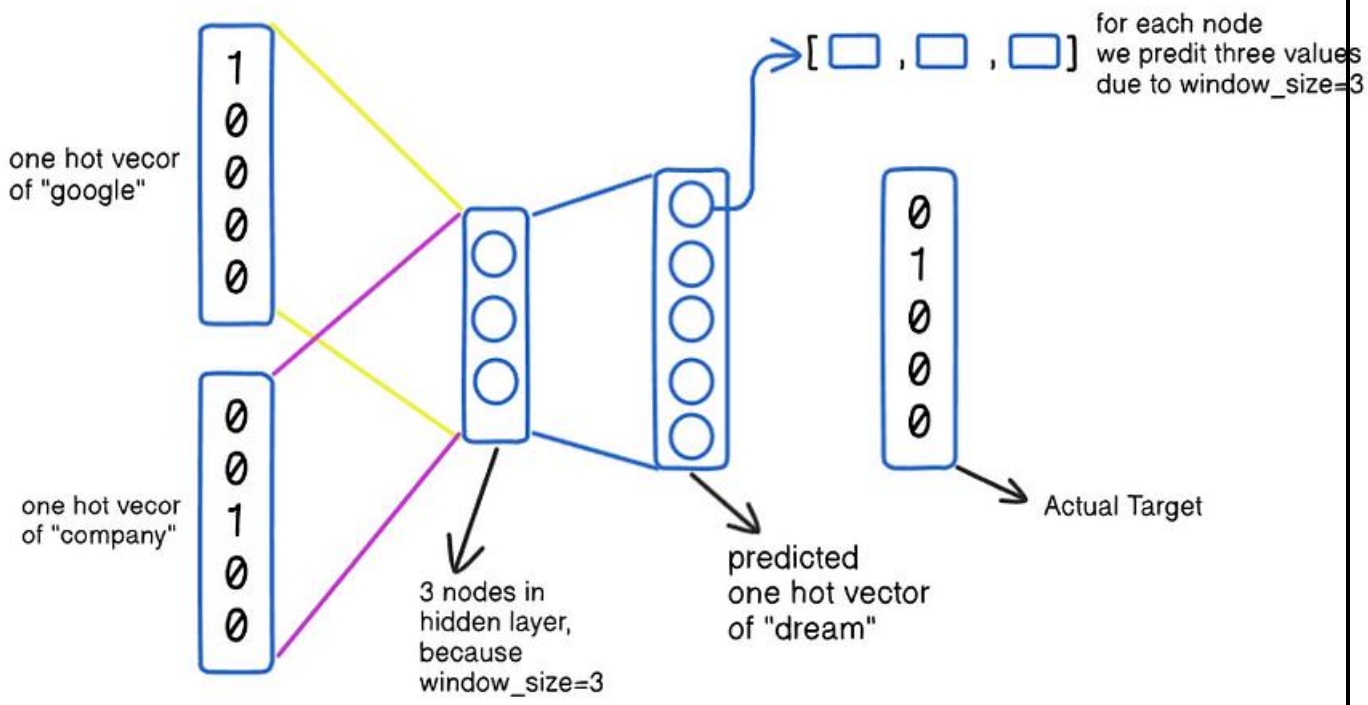
- Select the window size (window_size=3).
- The target is to predict the center word from context words (Surrounding words).
- We create the dataset where the context word is independent features and the center word is our output.



- Convert it into one hot encoding.

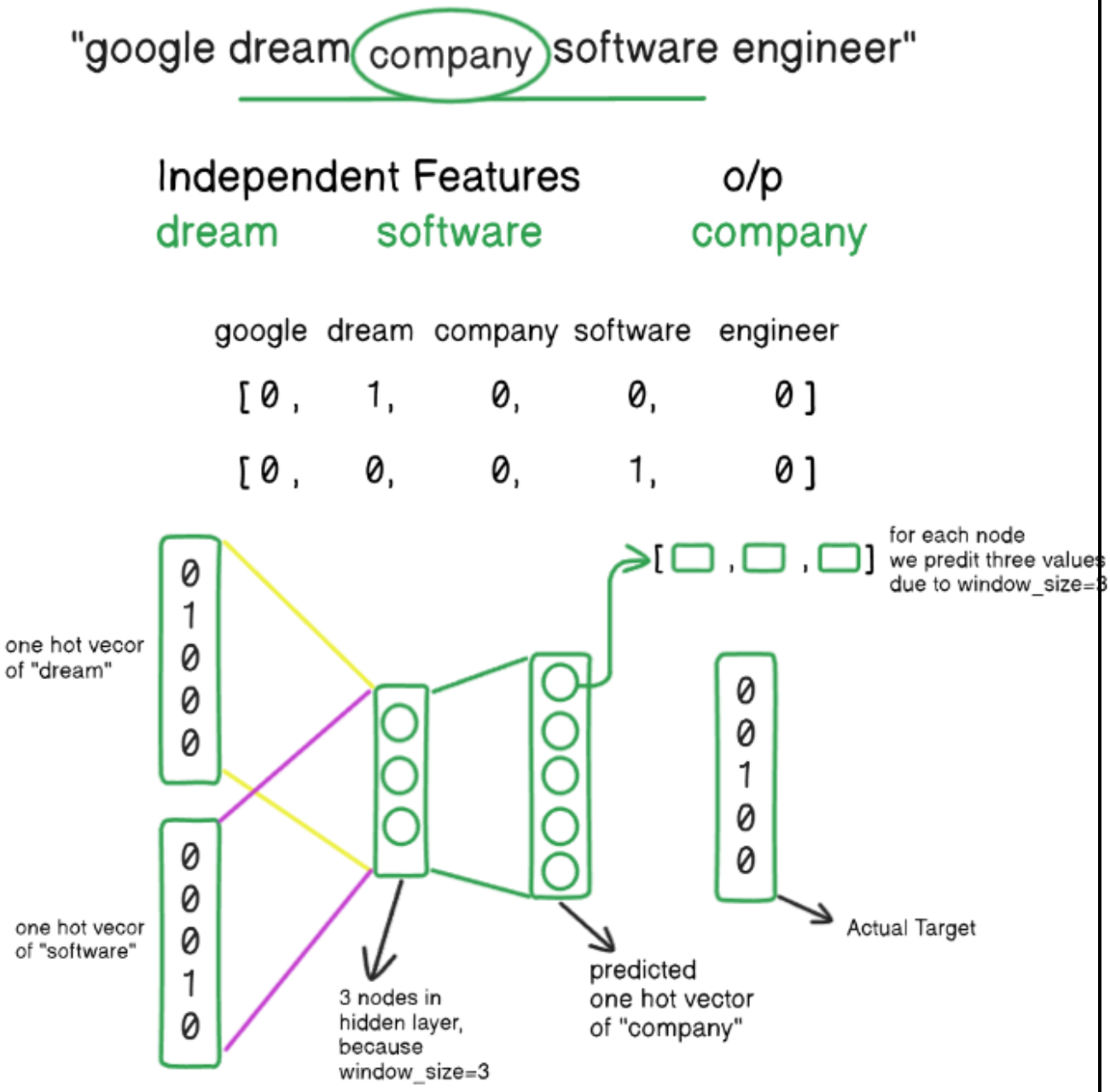
google	dream	company	software	engineer
[1,	0,	0,	0,	0]
[0,	0,	1,	0,	0]

- Next step, we pass it to the Neural Network



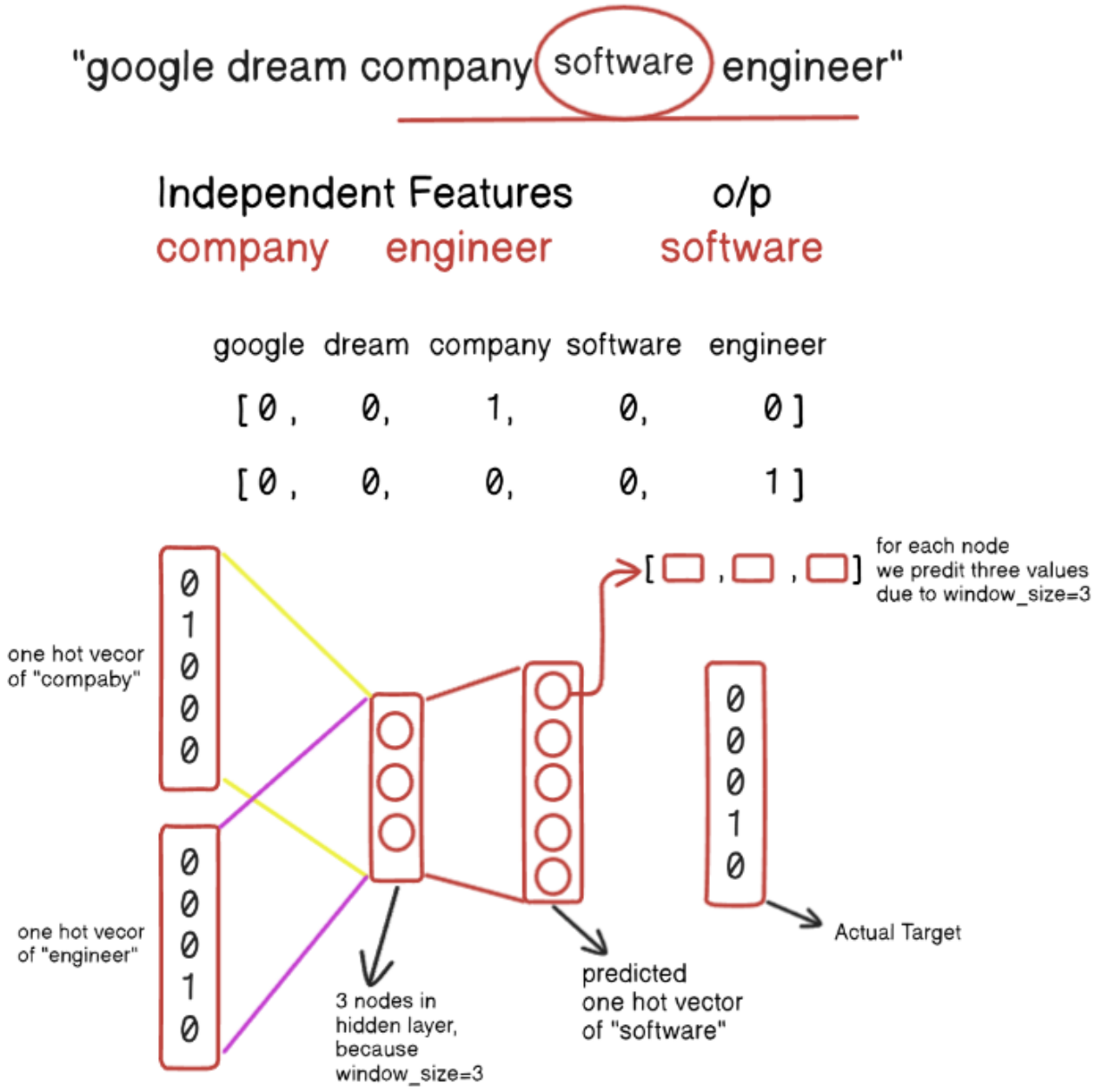
2nd Iteration:

- We go for the next three words



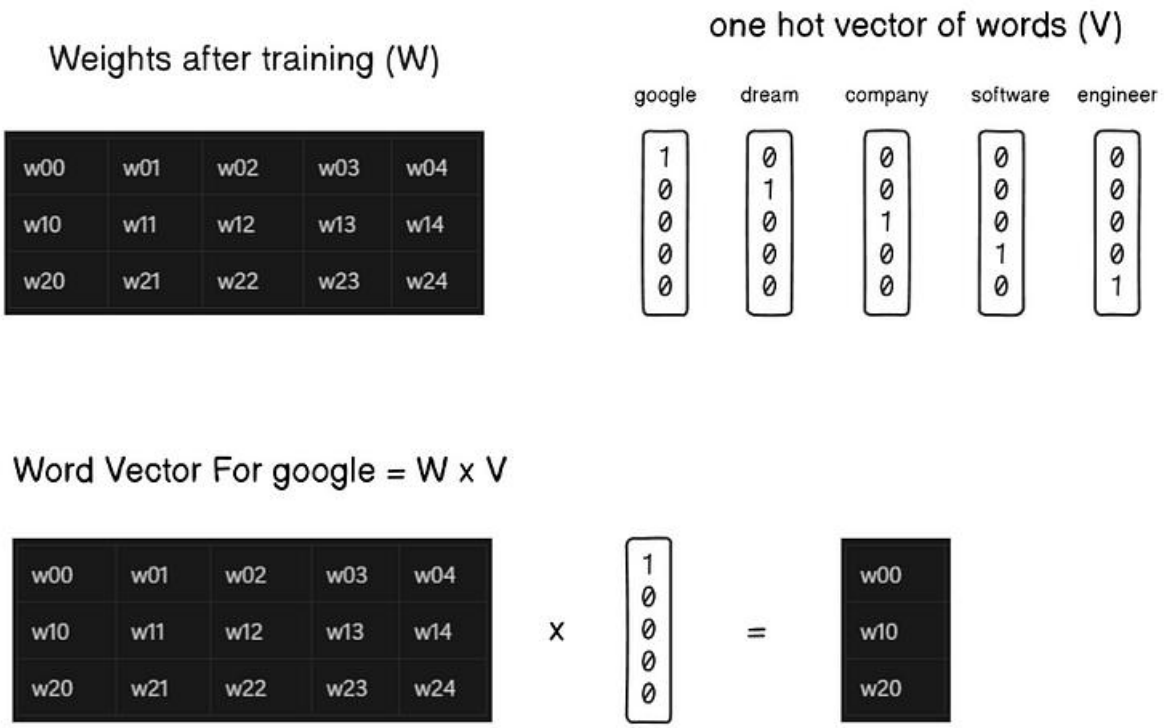
3rd Iteration:

- Finally, for the last three words the process is shown below:



Getting Word Embeddings:

- below is shown the process of getting word embeddings:



Skip-Gram

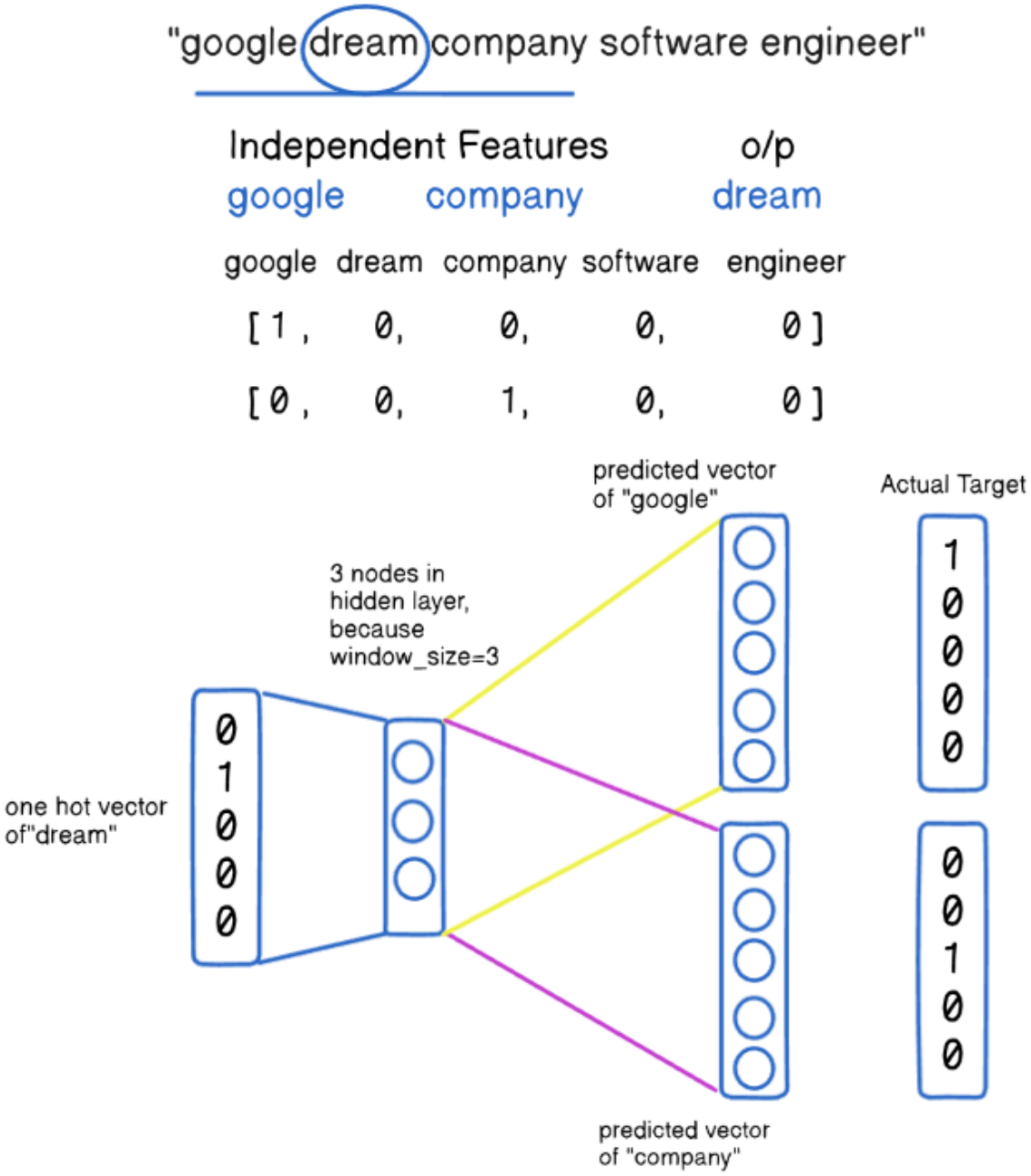
- Predict the surrounding context words given a target word.
- The model takes the target word and tries to predict each of the context words within a window.
- Performs better on smaller datasets and can capture more complex relationships between words.

Practical Example:

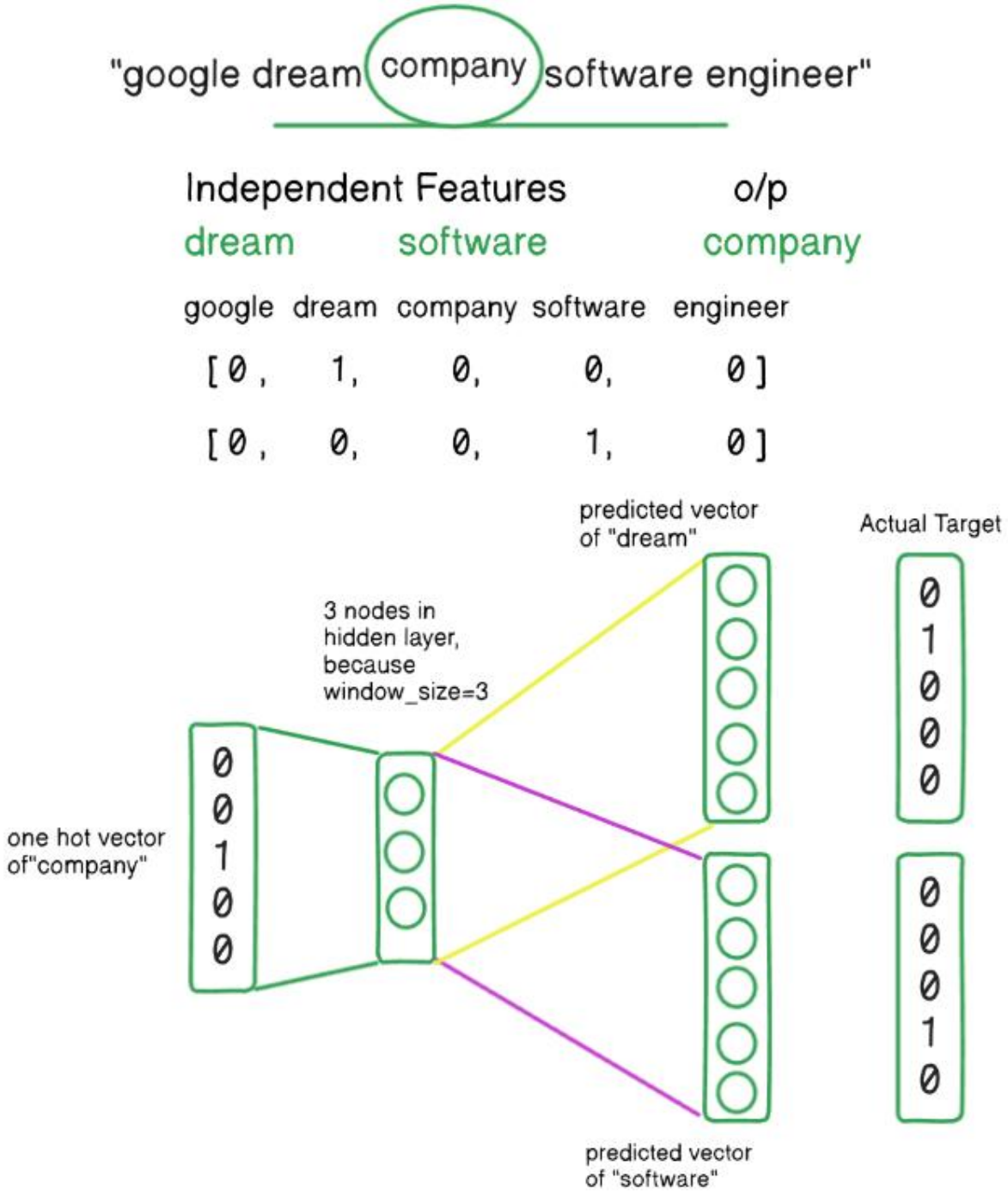
- We will use the above example:

1st Iteration:

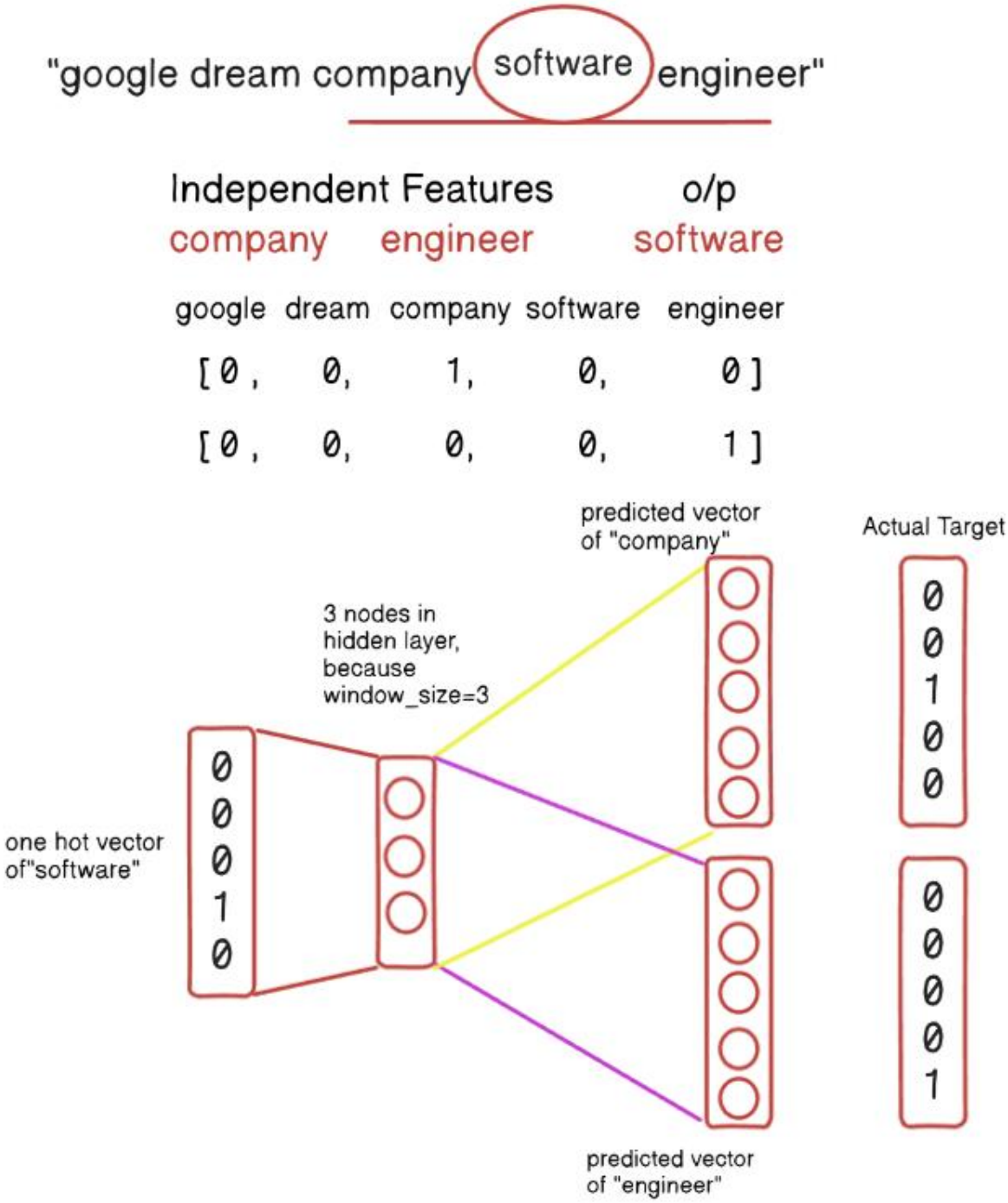
- Here, our input is our output value, and we predict the surrounding words.



2nd Iteration:



3rd Iteration:



Extensions and Alternatives

Several models build on or improve Word2Vec:

1. **GloVe (Global Vectors for Word Representation)**: Combines global word co-occurrence statistics with local context-based learning.
2. **FastText**: Extends Word2Vec by representing words as n-grams of characters, improving representations for rare words.

3. **ELMo (Embeddings from Language Models)**: Uses deep, contextualized word representations.
4. **BERT (Bidirectional Encoder Representations from Transformers)**: Uses transformers for contextualized word embeddings, considering both left and right context.

Applications

1. **Semantic Similarity**: Measuring similarity between words.
2. **Text Classification**: Improving feature representation for classification tasks.
3. **Machine Translation**: Enhancing translation quality by providing better word embeddings.
4. **Information Retrieval**: Improving search results by understanding word semantics.
5. **Recommendation Systems**: Enhancing recommendations by understanding user preferences and item descriptions.

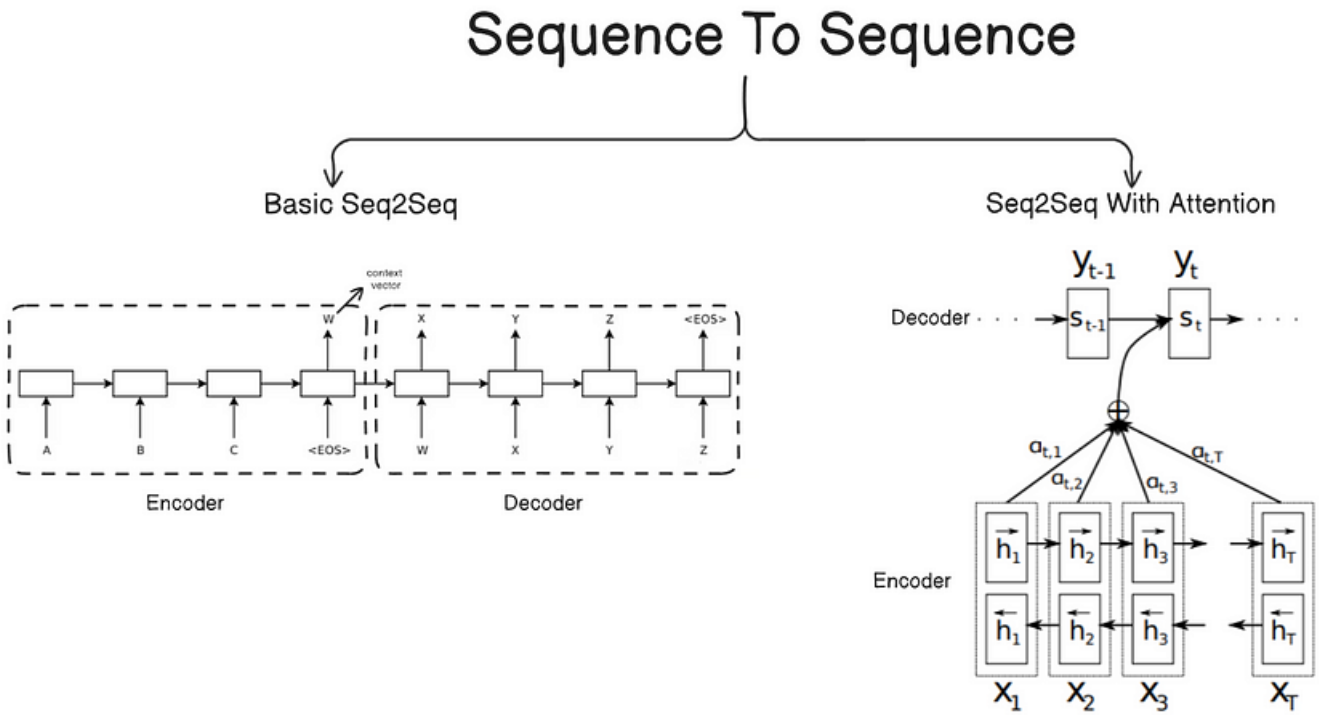
Limitations

1. **Context Independence**: It doesn’t consider the order of words or their syntactic roles.
2. **Out-of-Vocabulary Words**: It cannot handle words that were not present in the training corpus.

3. **Fixed Embedding Size**: All words are represented by fixed-size vectors, regardless of their frequency or importance.

Seq2Seq (Encoder/Decoder) And Attention Mechanism

- Sequence-to-sequence (seq2seq) models are a type of neural network architecture used primarily for tasks where the input and output are sequences of data, which can vary in length. They have been widely used in various applications, including machine translation, text summarization, and speech recognition.



Key Components

1. **Encoder**: The encoder processes the input sequence and compresses it into a fixed-size context vector (also called a thought vector). It typically consists of Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, or Gated Recurrent Units (GRUs).
2. **Decoder**: The decoder takes the context vector produced by the encoder and generates the output sequence. Like the encoder, it is usually an RNN, LSTM, or GRU.
3. **Attention Mechanism**: An enhancement to the basic seq2seq model, the attention mechanism allows the model to focus on different parts of

the input sequence at each step of the output generation. This helps in handling longer sequences more effectively.

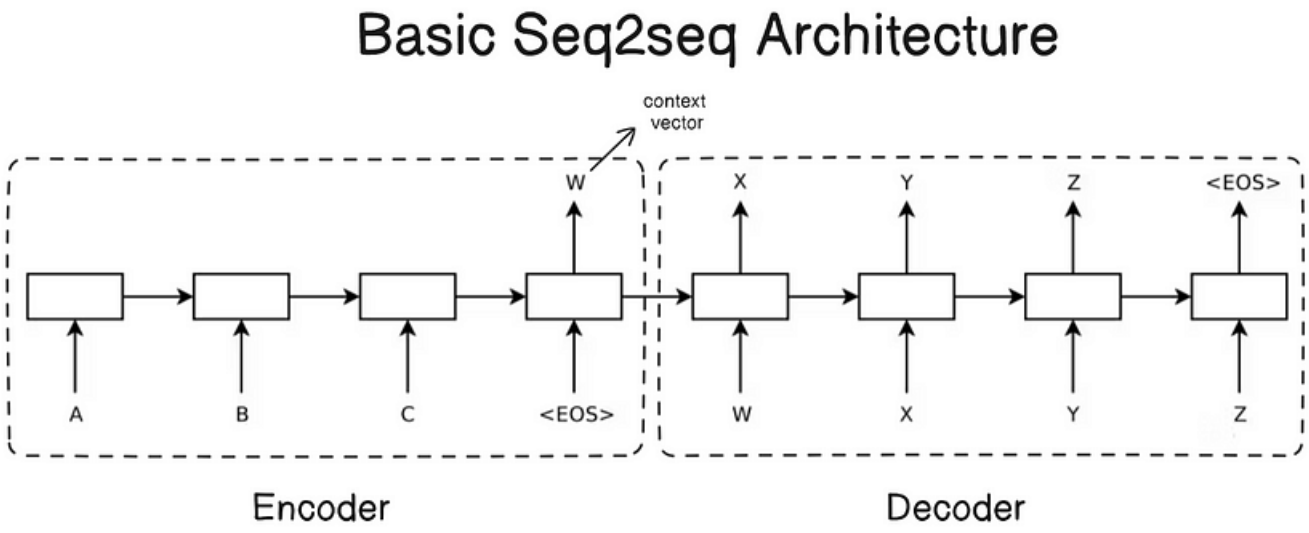
Architecture

1. Basic Seq2Seq Model
2. Seq2Seq With Attention

Basic Seq2Seq Model

Research Paper: *Sequence to Sequence Learning with Neural Networks*

1. **Input Sequence**: A sequence of tokens (e.g., words in a sentence) is fed into the encoder.
2. **Encoder**: The encoder processes this sequence and outputs a context vector.
3. **Context Vector**: This vector captures the information from the entire input sequence.
4. **Decoder**: The context vector is fed into the decoder, which generates the output sequence token by token.
5. **Output Sequence**: The generated sequence is the model's final output.



- Our model reads an input sentence "ABC" and produces "XYZ" as the output sentence.
- The model stops making predictions after outputting the end-of-sentence token.
- Note that the LSTM reads the input sentence in reverse because doing so introduces many short-term dependencies in the data that make the optimization problem much easier.

Model:

$$h_{<t>} = sigm(W_{hx}X + W_{hh}h_{<t-1>})$$
$$y_{<t>} = W_{yh}h_{<t>}$$

- The goal of the LSTM is to estimate the conditional probability p(y1, . . . , yT′ |x1, . . . , xT) where (x1, . . . , xT) is an input sequence and y1, . . . , yT′ is its corresponding output sequence whose length T′ may differ from T.
- The LSTM computes this conditional probability by first obtaining the fixed dimensional representation v of the input sequence (x1, . . . , xT) given by the last hidden state of the LSTM, and then computing the probability of y1, . . . , yT′ with a standard LSTM-LM formulation whose initial hidden state is set to the representation v of x1, . . . , xT :

$$p(y_1, ..., yT'|x_1, ..., xT) = \Pi_{i=1}^{T'} p(y_t|v, y_1, ..., y_{t-1})$$

- In this equation, each p(yt|v, y1, . . . , yt−1) distribution is represented with a softmax over all the words in the vocabulary. We use the LSTM formulation from Graves.
- Note that we require that each sentence ends with a special end-of-sentence symbol “<EOS>”, which enables the model to define a distribution over sequences of all possible lengths. The overall scheme is outlined in the above picture, where the shown LSTM computes the representation of “A”, “B”, “C”, “<EOS>” and then uses this representation to compute the probability of “W”, “X”, “Y”, “Z”, “<EOS>”.

Research Paper Key Points About The Model:

Our actual models differ from the above description in three important ways. First, we used two different LSTMs: one for the input sequence and another for the output sequence, because doing so increases the number model parameters at negligible computational cost and makes it natural to train the LSTM on multiple language pairs simultaneously. Second, we found that deep LSTMs significantly outperformed shallow LSTMs, so we chose an LSTM with four layers.

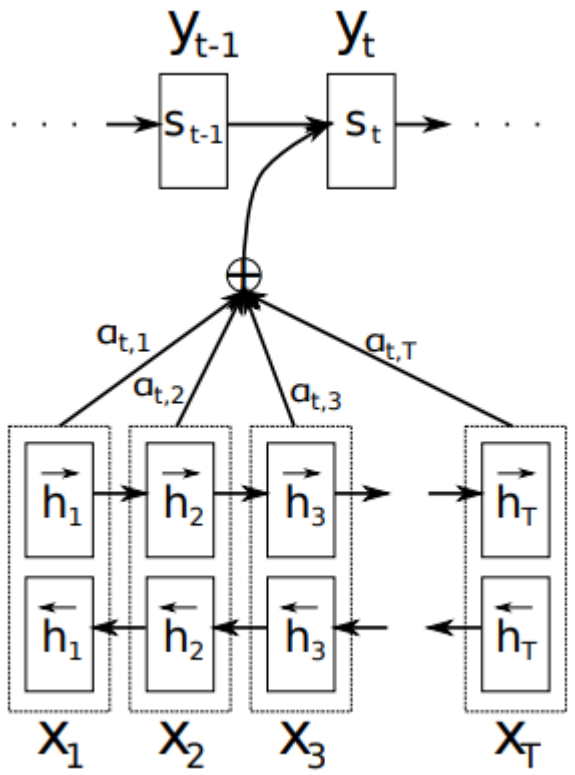
Third, we found it extremely valuable to reverse the order of the words of the input sentence. So for example, instead of mapping the sentence a, b, c to the sentence α, β, γ, the LSTM is asked to map c, b, a to α, β, γ, where α, β, γ is the translation of a, b, c. This way, a is in close

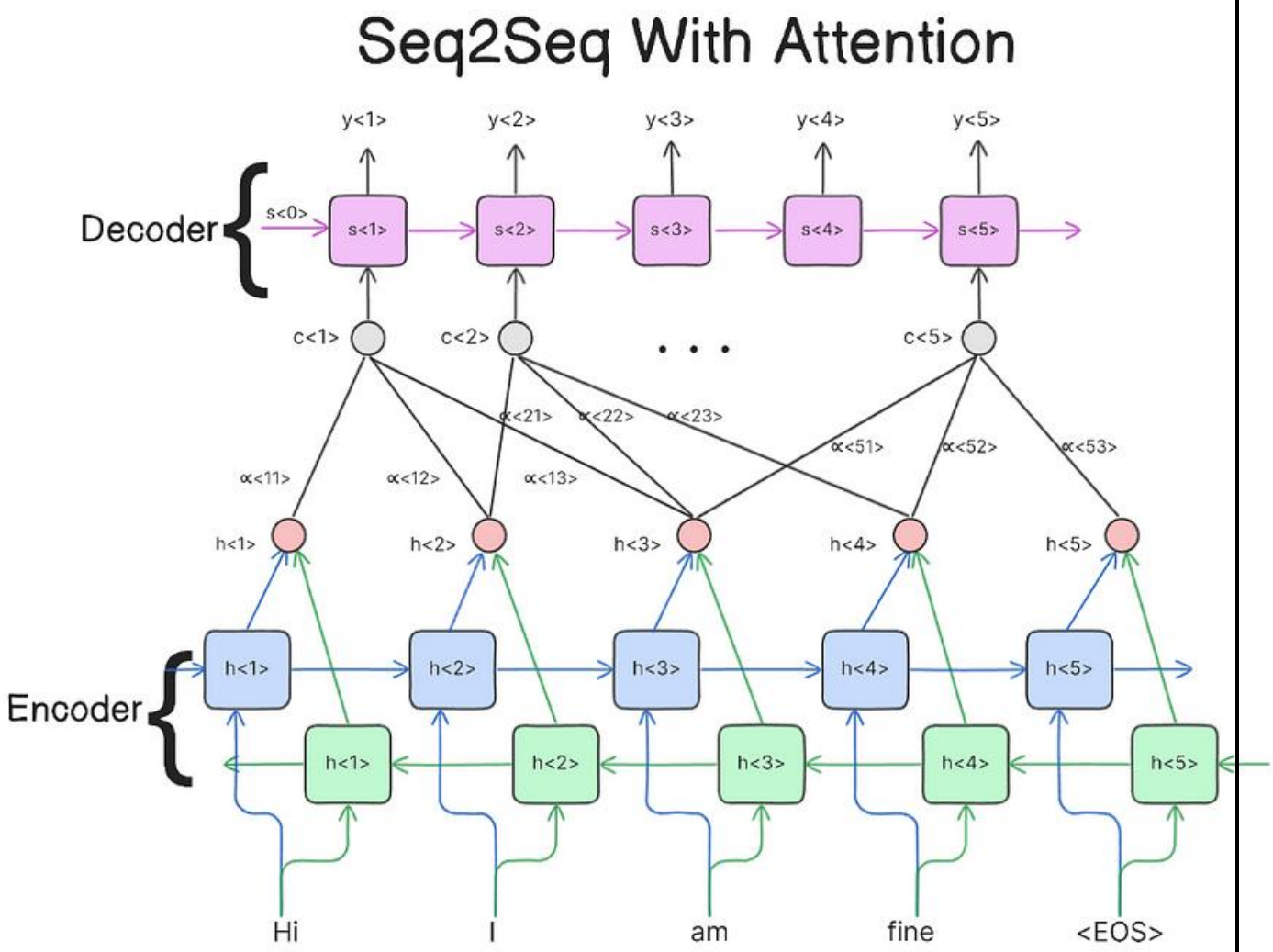
proximity to α, b is fairly close to β, and so on, a fact that makes it easy for SGD to "establish communication" between the input and the output. We found this simple data transformation to greatly boost the performance of the LSTM.

Seq2Seq with Attention

Research Paper: *NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE*

1. **Input Sequence**: Same as above.
2. **Encoder**: Processes the sequence and outputs a sequence of hidden states.
3. **Attention Layer**: At each decoding step, the attention mechanism computes a context vector as a weighted sum of the encoder's hidden states. The weights are determined based on the relevance of each hidden state to the current decoding step.
4. **Decoder**: Uses the context vector from the attention layer to generate each token of the output sequence.





- The context vector c_i depends on a sequence of annotations (h_1, · · · , h_Tx) to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i-th word of the input sequence.
- In the above image, Tx=3
- The context vector c_i is, then, computed as a weighted sum of these annotations h_i:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

- The weight αij of each annotation h_j is computed by:

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{Tx} exp(e_{ik})},$$

Where:

$$e_{ij} = a(s_{i-1}, h_j)$$

is an alignment model which scores how well the inputs around position j and the output at position i match. The score is based on the RNN hidden state s_i−1 and the j-th annotation h_j of the input sentence. We parametrize the alignment model a as a feedforward neural network which is jointly trained with all the other components of the proposed system.

Implementation Of seq2seq Model

- Seq2seq models are trained using supervised learning, where the training data consists of input-output sequence pairs. The objective is to minimize the difference between the generated output sequence and the target sequence.
- The most commonly used loss function is the categorical cross-entropy loss, calculated for each token in the output sequence.

Here’s a simplified example using TensorFlow and Keras:

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense

# Encoder
encoder_inputs = Input(shape=(None, num_encoder_tokens))
encoder = LSTM(latent_dim, return_state=True)
encoder_outputs, state_h, state_c = encoder(encoder_inputs)
encoder_states = [state_h, state_c]
# Decoder
decoder_inputs = Input(shape=(None, num_decoder_tokens))
decoder_lstm = LSTM(latent_dim, return_sequences=True,
return_state=True)
decoder_outputs, _, _ = decoder_lstm(decoder_inputs,
initial_state=encoder_states)
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)
# Seq2Seq Model
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
# Compile the model
model.compile(optimizer='rmsprop', loss='categorical_crossentropy')
```

```
# Fit the model
model.fit([encoder_input_data, decoder_input_data],
decoder_target_data,
        batch_size=batch_size,
        epochs=epochs,
        validation_split=0.2)
```

Applications

1. **Machine Translation**: Translating text from one language to another.
2. **Text Summarization**: Creating a concise summary of a longer text.
3. **Speech Recognition**: Converting spoken language into text.
4. **Chatbots and Conversational Agents**: Generating responses in a dialogue system.
5. **Image Captioning**: Generating descriptive text for a given image.

Challenges and Solutions

1. **Handling Long Sequences**: Long input sequences can be difficult for basic RNNs to manage. Attention mechanisms and more advanced architectures like Transformers address this issue.
2. **Data Sparsity**: For tasks like machine translation, a large amount of parallel text data is required.
3. **Training Time**: Seq2seq models can be computationally expensive to train.

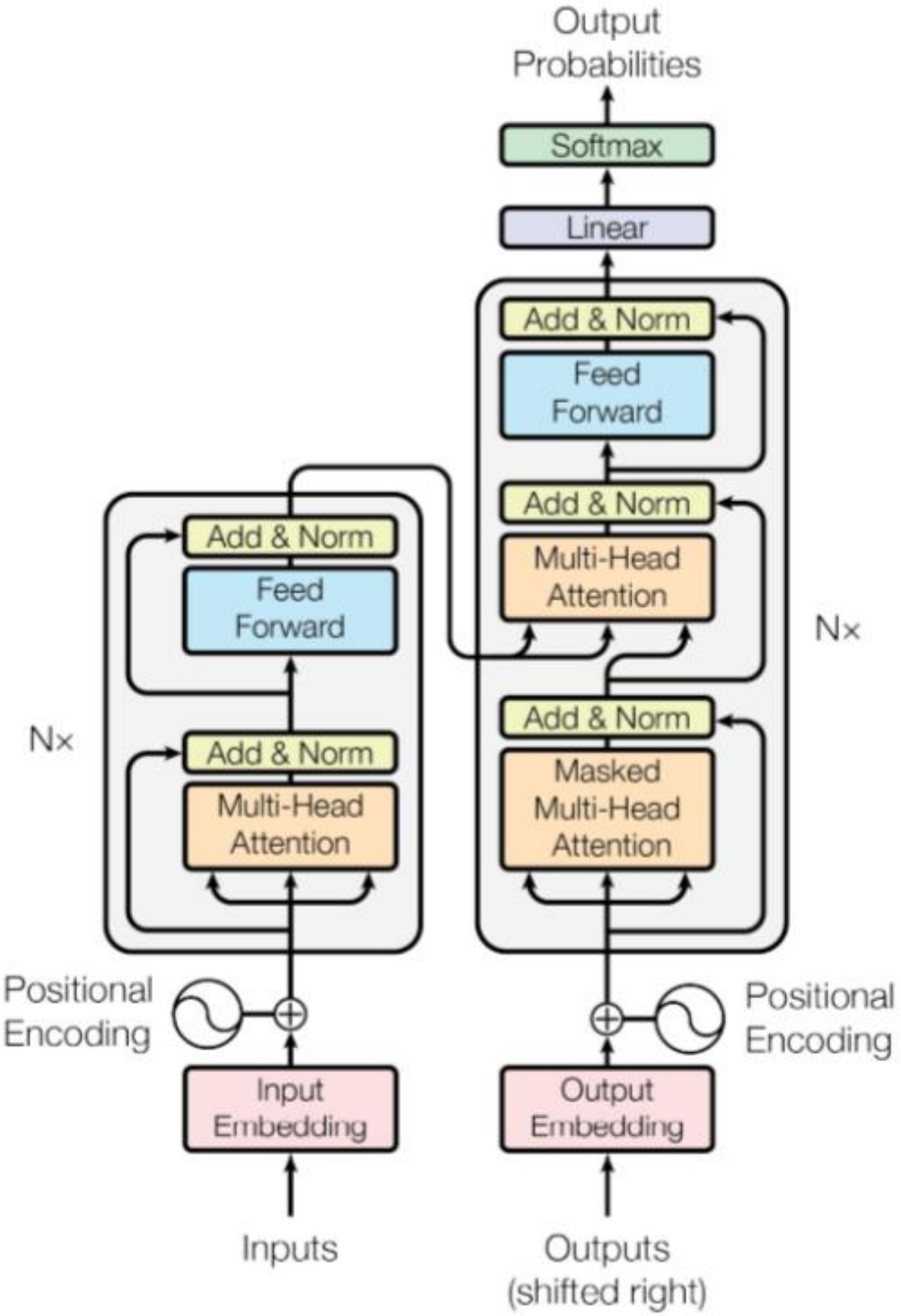
Advanced Models

1. **Transformer**: A more recent architecture that relies entirely on attention mechanisms and has significantly improved performance on many seq2seq tasks.
2. **BERT/GPT**: Pre-trained models that can be fine-tuned for specific seq2seq tasks.

Transformers

Transformers, in the context of machine learning and artificial intelligence, refer to a type of deep learning model architecture designed primarily for natural language processing (NLP) tasks. They have revolutionized the field by enabling more effective training and performance on a variety of tasks such as translation, summarization, and question-answering.

Transformer Model architecture



1. Introduction and History

- **Origins**: The transformer model was introduced in the paper “Attention Is All You Need” by Vaswani et al., published by Google Brain in 2017.
- **Impact**: Transformers have replaced recurrent neural networks (RNNs) and long short-term memory (LSTM) networks in many NLP tasks due to their efficiency and performance.

2. Architecture Overview

- **Attention Mechanism**: The core innovation of transformers is the self-attention mechanism, which allows the model to weigh the importance of different words in a sentence.
- **Encoder-Decoder Structure**: The original transformer model has an encoder-decoder structure. The encoder processes the input sequence, and the decoder generates the output sequence.
- **Parallelization**: Transformers can process all words in a sentence simultaneously, unlike RNNs which process sequentially, making training faster.

3. Components of a Transformer

- **Input Embeddings**: Converts words into vectors of real numbers.
- **Positional Encoding**: Adds information about the position of each word in the sentence, since transformers do not inherently understand order.
- **Multi-Head Self-Attention**: Allows the model to focus on different parts of the sentence simultaneously by using multiple attention mechanisms in parallel.
- **Feed-forward Neural Networks**: Applied independently to each position to introduce non-linearity.
- **Layer Normalization**: Normalizes inputs to each layer to stabilize and accelerate training.
- **Residual Connections**: Helps with gradient flow and allows for deeper models by adding the input of each layer to its output.

4. Self-Attention Mechanism

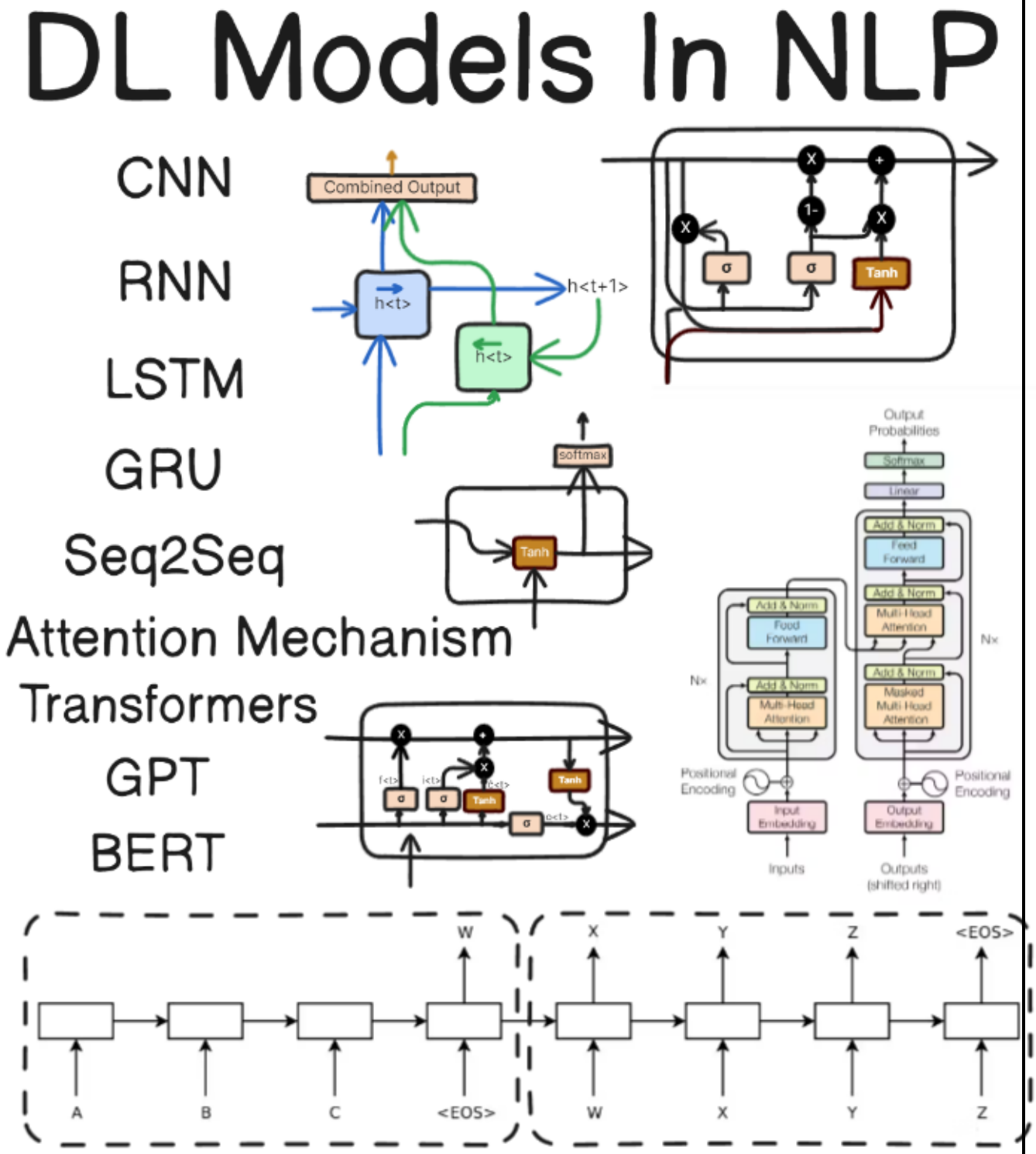
- **Calculation**: Self-attention involves computing three vectors for each word — Query (Q), Key (K), and Value (V).
- **Scaled Dot-Product**: The dot product of the query and key vectors determines the attention scores, which are then scaled and passed through a SoftMax function to get attention weights.
- **Weighted Sum**: The attention weights are used to compute a weighted sum of the value vectors, producing the final output.

5. Transformer Practical Explanation

- Our goal is to translate French sentences into English.

DL Models Used In NLP (CNN, RNNs, Seq-to-Seq, Transformers)

Deep learning models have revolutionized Natural Language Processing (NLP) by enabling significant advancements in tasks such as machine translation, sentiment analysis, text generation, and more.



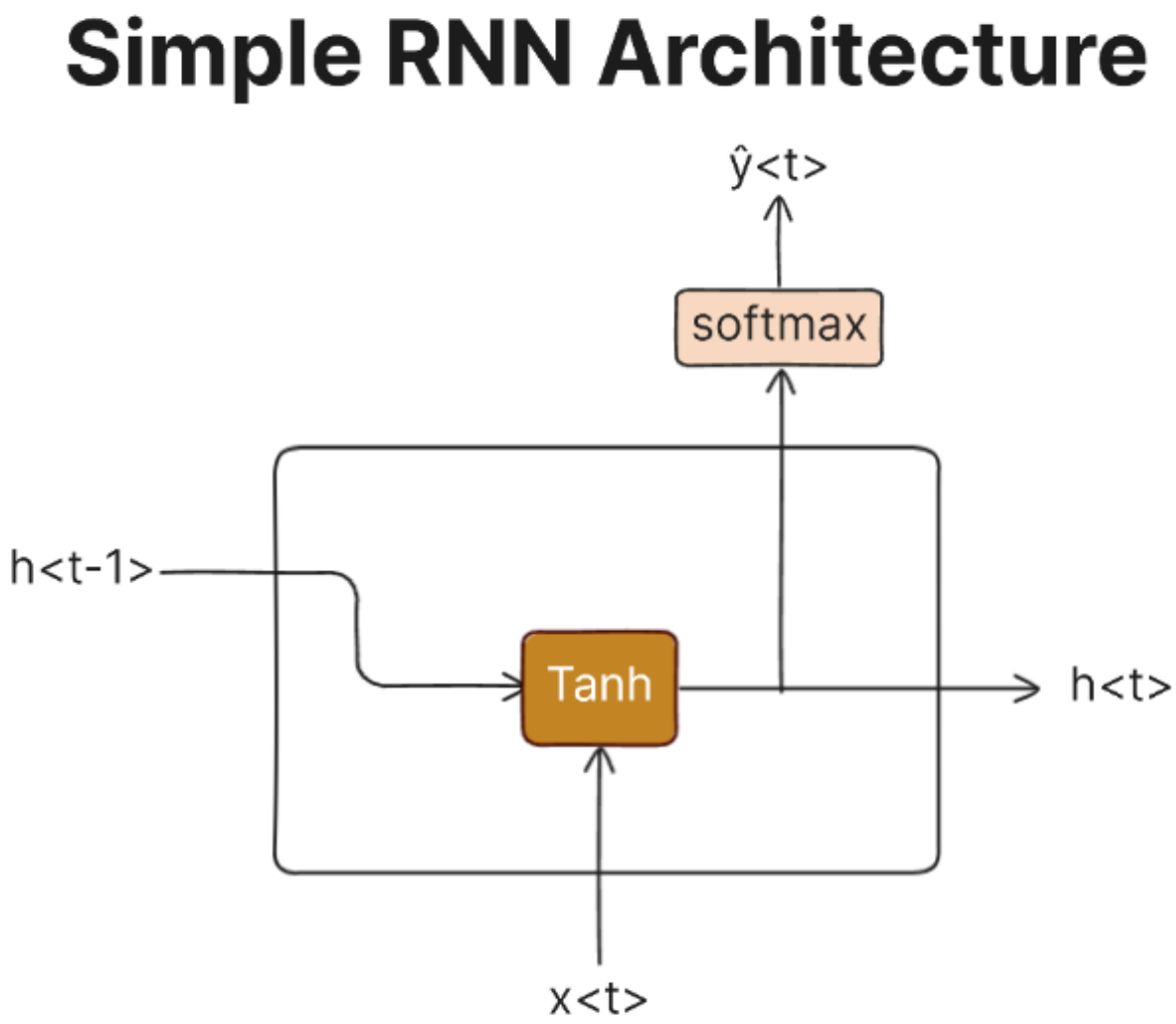
1. Convolutional Neural Networks (CNNs)

- Although CNNs are primarily used in image processing, they are also effective for certain NLP tasks such as text classification, where they capture local patterns in text.

[Link For More Details]

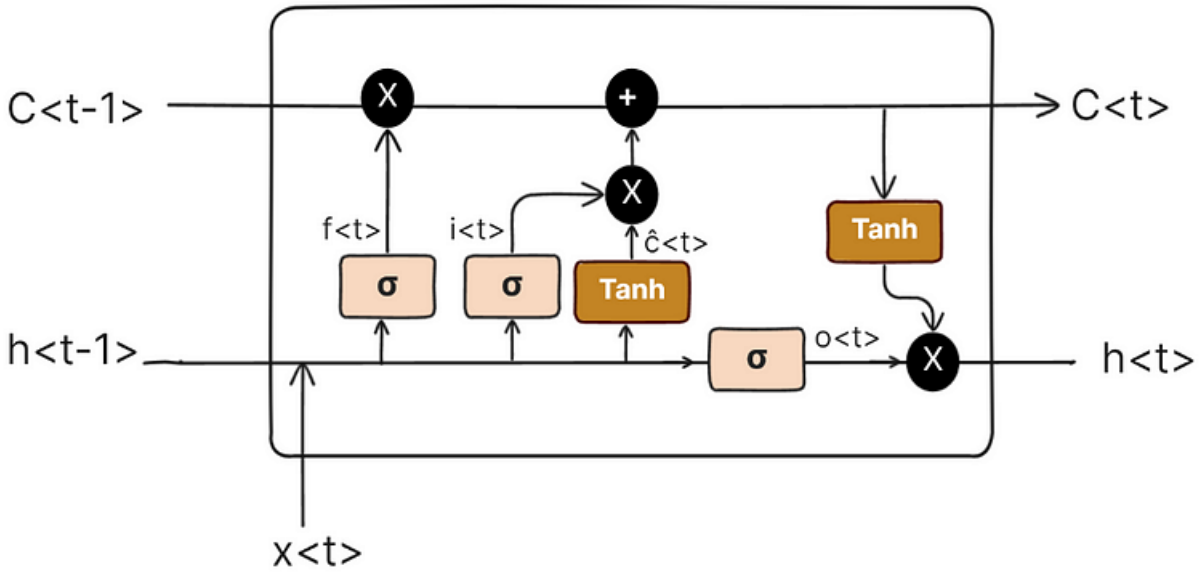
2. Recurrent Neural Networks (RNNs)

- **Simple RNNs**: A basic form of RNNs, where the output from the previous step is fed as input to the current step.
  [Link For More Details]



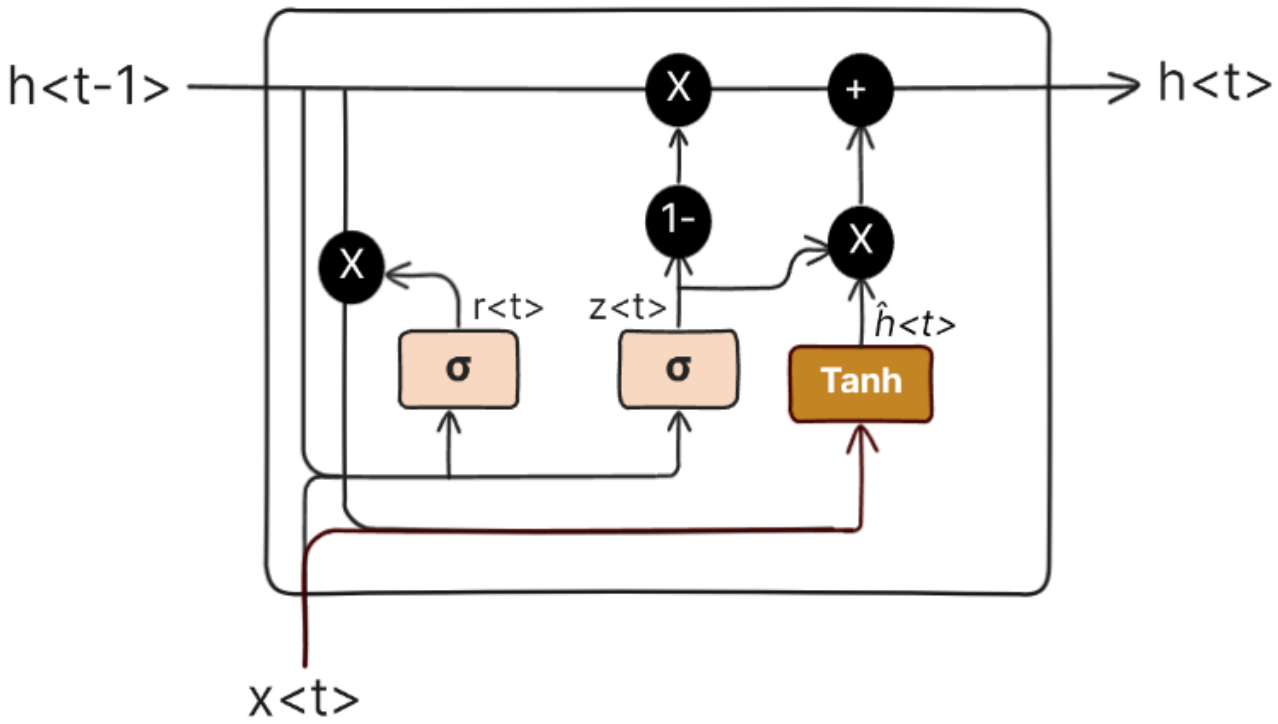
- **Long Short-Term Memory (LSTM)**: An advanced version of RNNs designed to remember information for long periods, solving the vanishing gradient problem.
  [Link For More Details]

LSTM Architecture



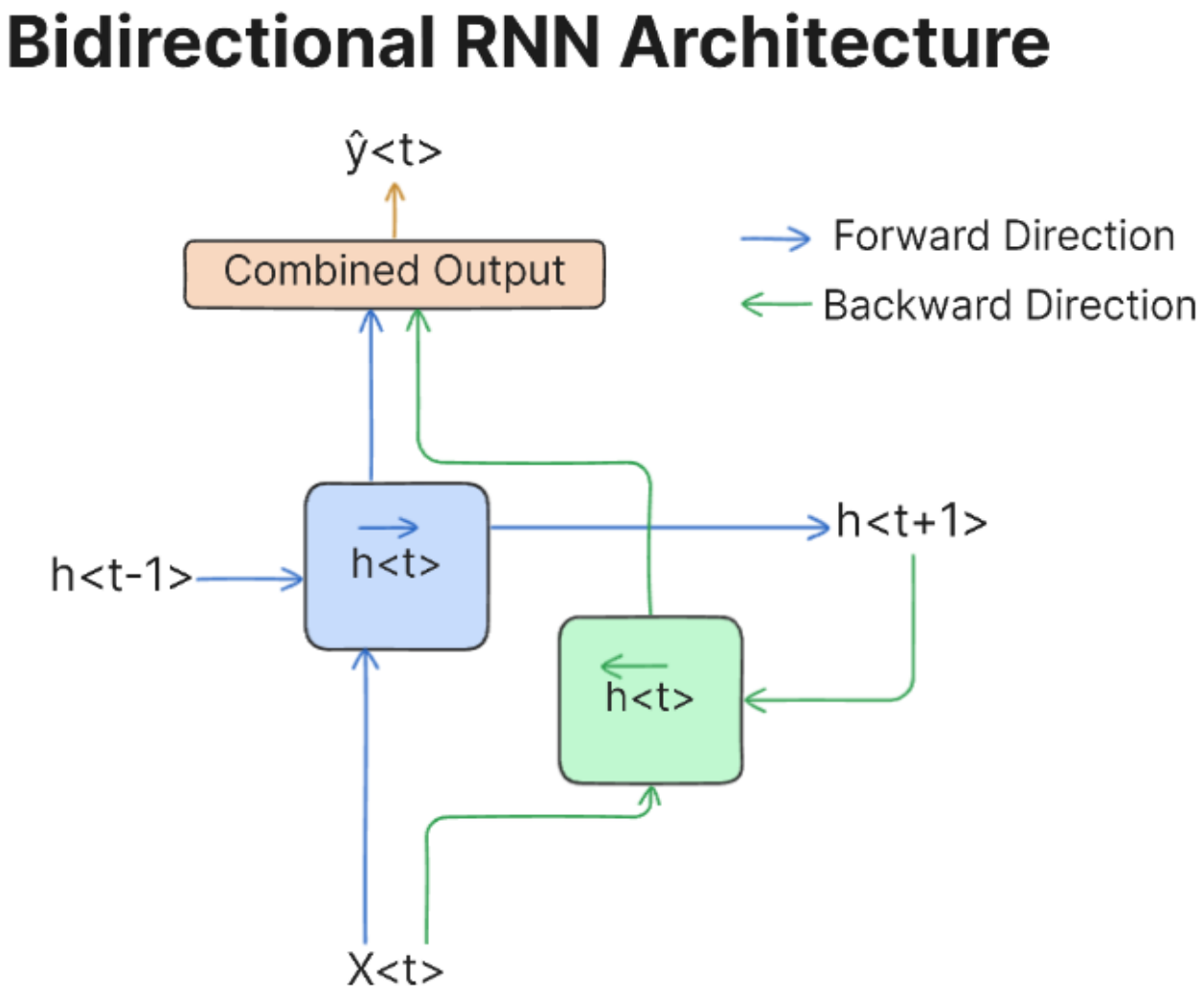
- **Gated Recurrent Unit (GRU)**: Similar to LSTMs but with a simplified architecture, it is often faster to train.
  [Link For More Details]

GRU Architecture



- **Bidirectional RNN:** Data flow in both directions, forward and backward.

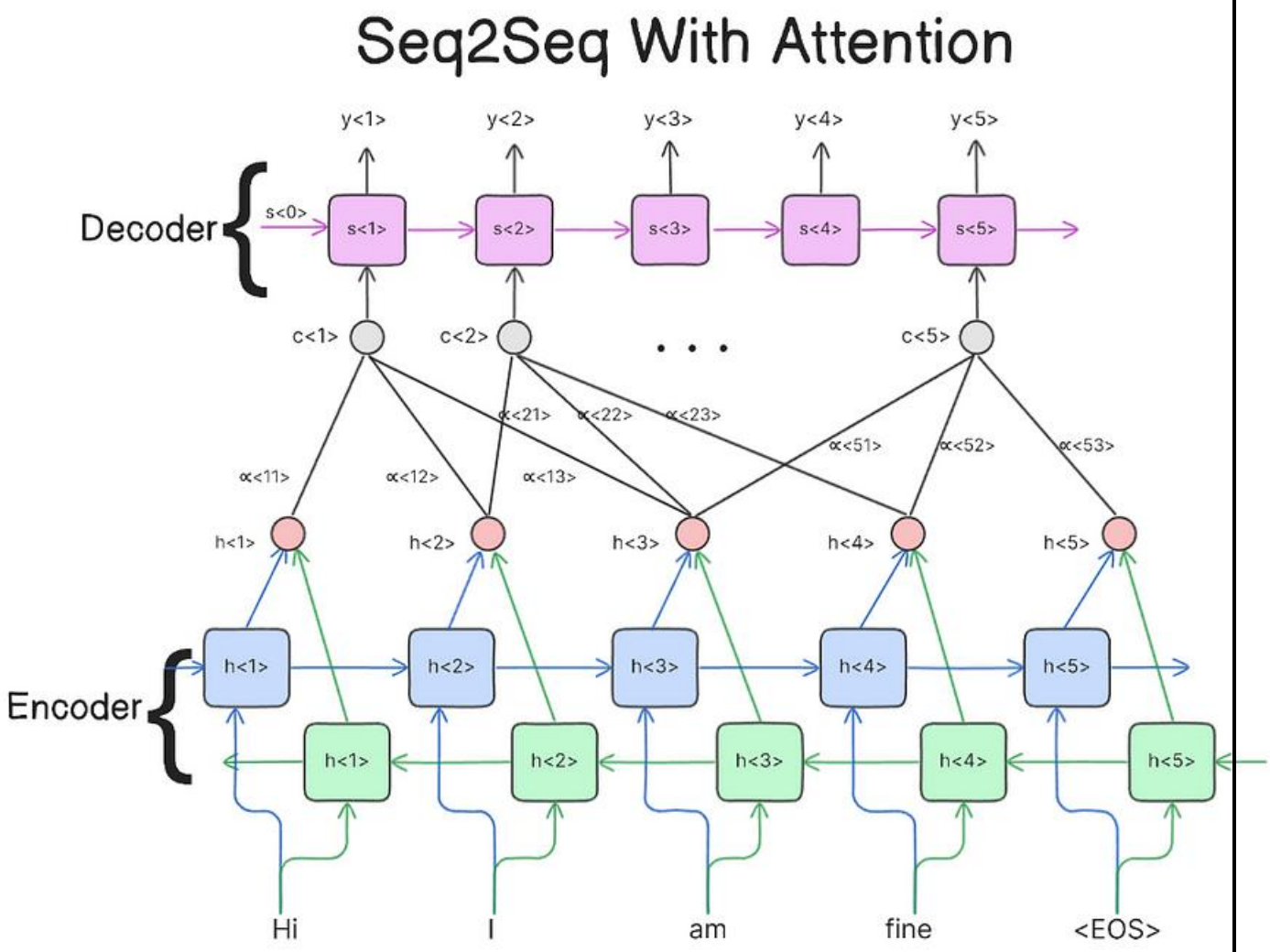
[Link For More Details]



3. Sequence-to-Sequence Models (Seq2Seq)

- Typically use encoder-decoder architectures, where the encoder processes the input sequence and the decoder generates the output sequence.
- We have basic seq2seq and seq2seq with attention mechanisms.
- Often paired with attention mechanisms for better performance.

[Link For More Details]



4. Transformers

- **Transformers**: The foundational architecture that uses self-attention mechanisms to handle long-range dependencies in text. Notable for parallelization and scalability.
  Link For More Details
- **BERT (Bidirectional Encoder Representations from Transformers)**: Pre-trained on vast amounts of data, BERT captures context from both directions (left-to-right and right-to-left).
- **GPT (Generative Pre-trained Transformer)**: Focuses on generating text by predicting the next word in a sentence, known for creating coherent and contextually relevant text.
- **T5 (Text-To-Text Transfer Transformer)**: Treats every NLP task as a text-to-text problem, simplifying the framework for multiple tasks.

Attention Mechanisms

- **Self-Attention**: Allows the model to weigh the importance of different words in a sentence when making predictions, which is crucial for understanding context.
- **Global Attention**: Used to focus on all parts of the input sentence when generating each part of the output, important for translation tasks.

Most Used Models

- The most widely used deep learning model in NLP today is the **Transformer** architecture, particularly its pre-trained variants. Among these, **BERT (Bidirectional Encoder Representations from Transformers)** and **GPT (Generative Pre-trained Transformer)** are the most prominent and extensively utilized.

1. BERT (Bidirectional Encoder Representations from Transformers)

- **Bidirectional Context**: Unlike previous models that read text either left-to-right or right-to-left, BERT reads in both directions, providing a deeper understanding of context.
- **Pre-training and Fine-tuning**: BERT is pre-trained on a large corpus of text and can be fine-tuned for specific tasks with relatively small datasets, making it versatile and easy to adapt to various NLP tasks.
- **Applications**: Widely used in tasks like question answering, named entity recognition, text classification, and more.

2. GPT (Generative Pre-trained Transformer)

- **Autoregressive Nature**: GPT generates text by predicting the next word in a sequence, which is powerful for text generation tasks.
- **Coherent Text Generation**: Known for its ability to generate coherent and contextually relevant text, making it ideal for applications like chatbots, content creation, and language translation.
- **Variants**: GPT-2 and GPT-3 (and the more recent GPT-4) have significantly improved capabilities, with GPT-3 and GPT-4 being known for their ability to perform a wide variety of tasks with minimal task-specific training data.